强化学习与博弈论

Reinforcement Learning and Game Theory

陈 旭

计算机学院



中山大學

SUN YAT-SEN UNIVERSITY

Chapter 3 Markov Decision Process

1 Markov Processes

2 Markov Reward Processes

3 Markov Decision Processes

Introduction to MDPs

- *Markov decision processes* formally describe an environment for reinforcement learning
- Where the environment is *fully observable*
- i.e. The current *state* completely characterises the process
- Almost all RL problems can be formalised as MDPs, e.g.
  - Optimal control primarily deals with continuous MDPs
  - Partially observable problems can be converted into MDPs

Markov Property

"The future is independent of the past given the present"

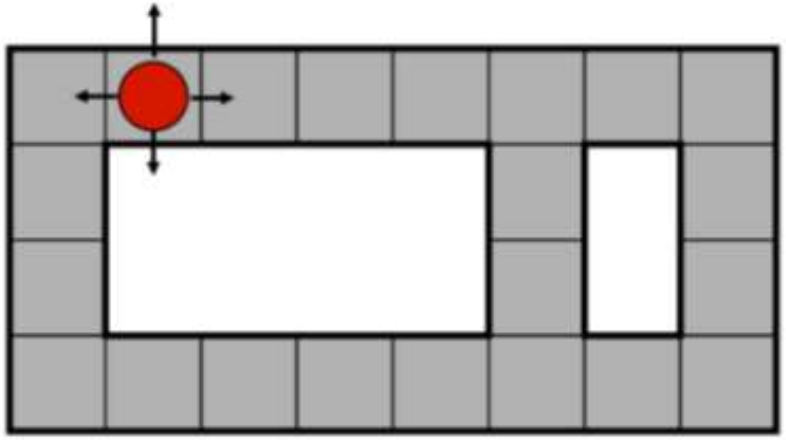
Definition

A state S_t is *Markov* if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, ..., S_t]$$

- The state captures all relevant information from the history
- Once the state is known, the history may be thrown away
- i.e. The state is a sufficient statistic of the future

Markov Property







State Transition Matrix

For a Markov state s and successor state s', the *state transition probability* is defined by

$$\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$$

State transition matrix $\mathcal{P}$ defines transition probabilities from all states s to all successor states s',

$$\mathcal{P} = \begin{matrix} \\ \textit{from} \end{matrix} \begin{matrix} \textit{to} \\ \end{matrix} \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix}$$

where each row of the matrix sums to 1.

Markov Process

A Markov process is a memoryless random process, i.e. a sequence of random states $S_1, S_2, \ldots$ with the Markov property.

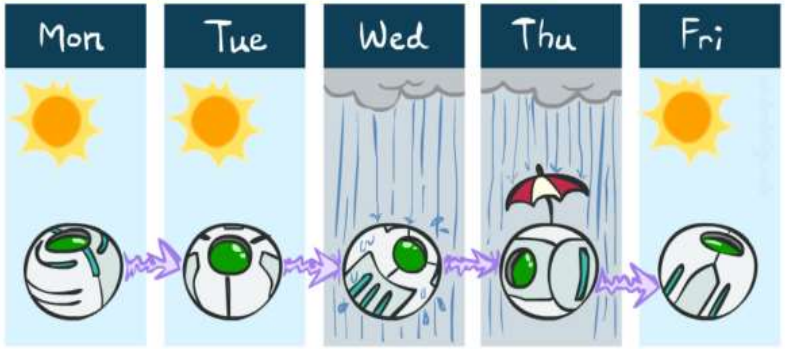
Definition

A *Markov Process* (or *Markov Chain*) is a tuple $\langle \mathcal{S}, \mathcal{P} \rangle$

- $\mathcal{S}$ is a (finite) set of states
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$

Example Markov Chain: Weather

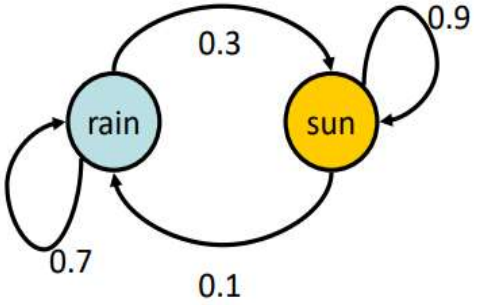
- States: X = {rain, sun}

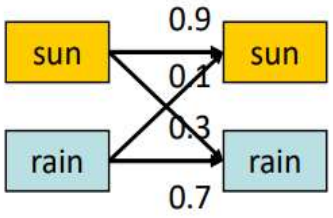


- Initial distribution: 1.0 sun

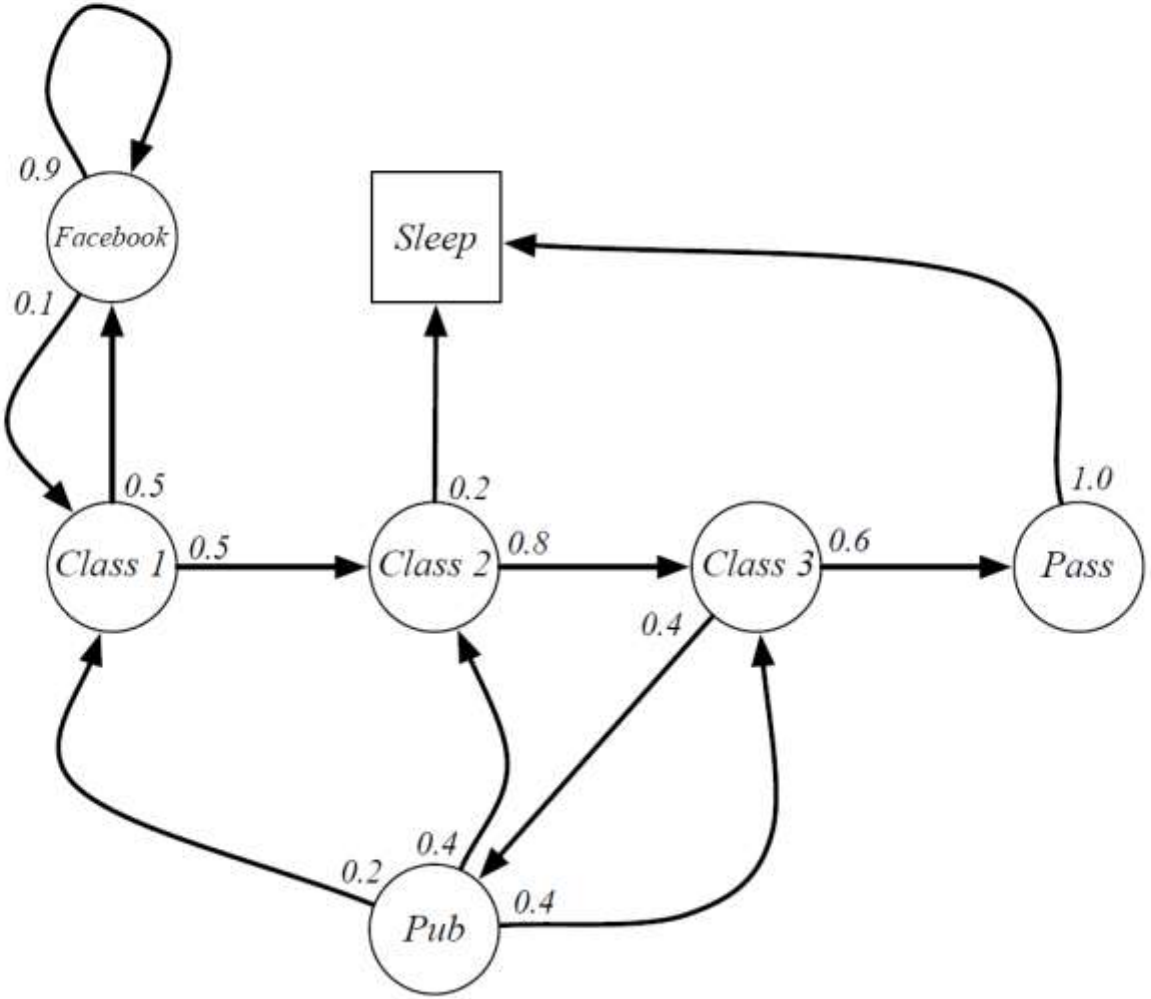
- $P(X_t \mid X_{t-1})$:

X_{t-1}	X_t	$P(X_t \mid X_{t-1})$
sun	sun	0.9
sun	rain	0.1
rain	sun	0.3
rain	rain	0.7

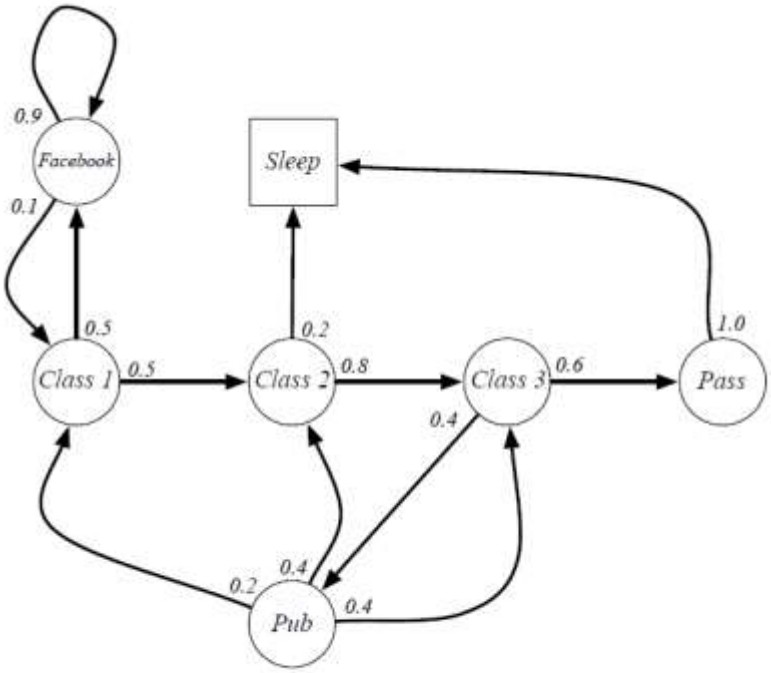




Example: Student Markov Chain



Example: Student Markov Chain Episodes

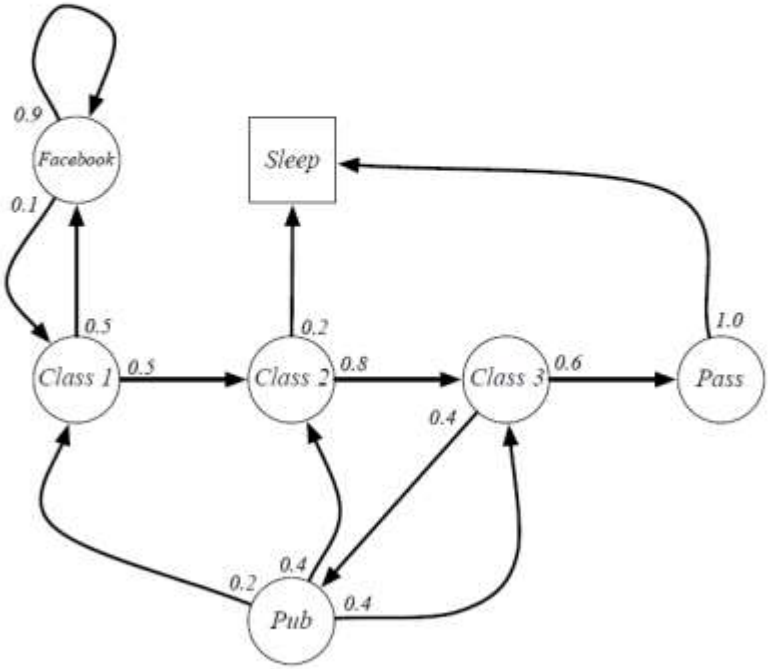


Sample episodes for Student Markov Chain starting from $S_1 = \text{C1}$

$$S_1, S_2, ..., S_T$$

- C1 C2 C3 Pass Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep

Example: Student Markov Chain Transition Matrix



$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{array}$$

Markov Reward Process

A Markov reward process is a Markov chain with values.

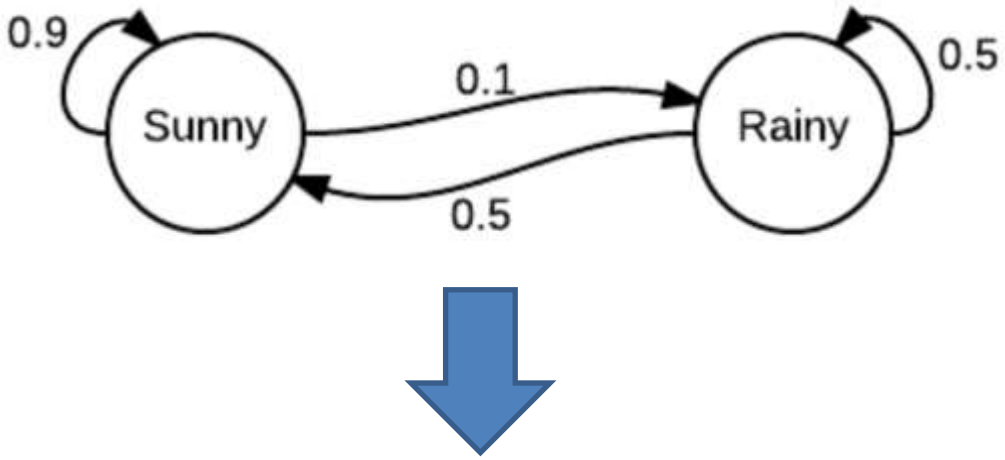
Definition

A *Markov Reward Process* is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

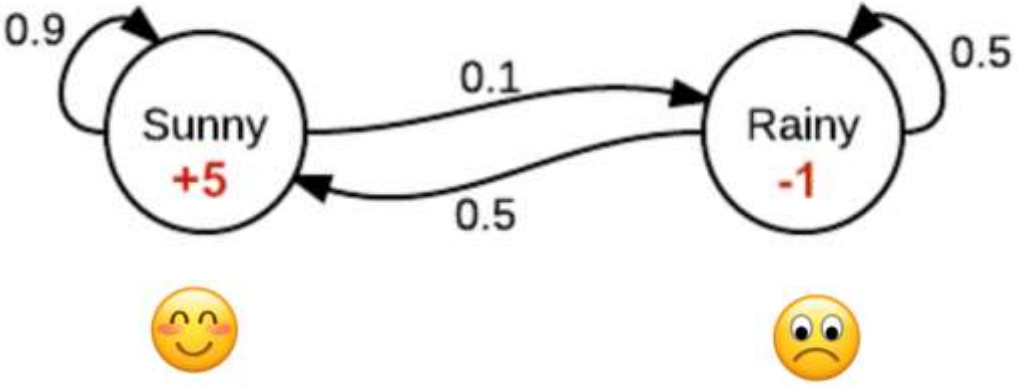
- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s = \mathbb{E}[R_{t+1} \mid S_t = s]$
- γ is a discount factor, $\gamma \in [0, 1]$

Markov Reward Process

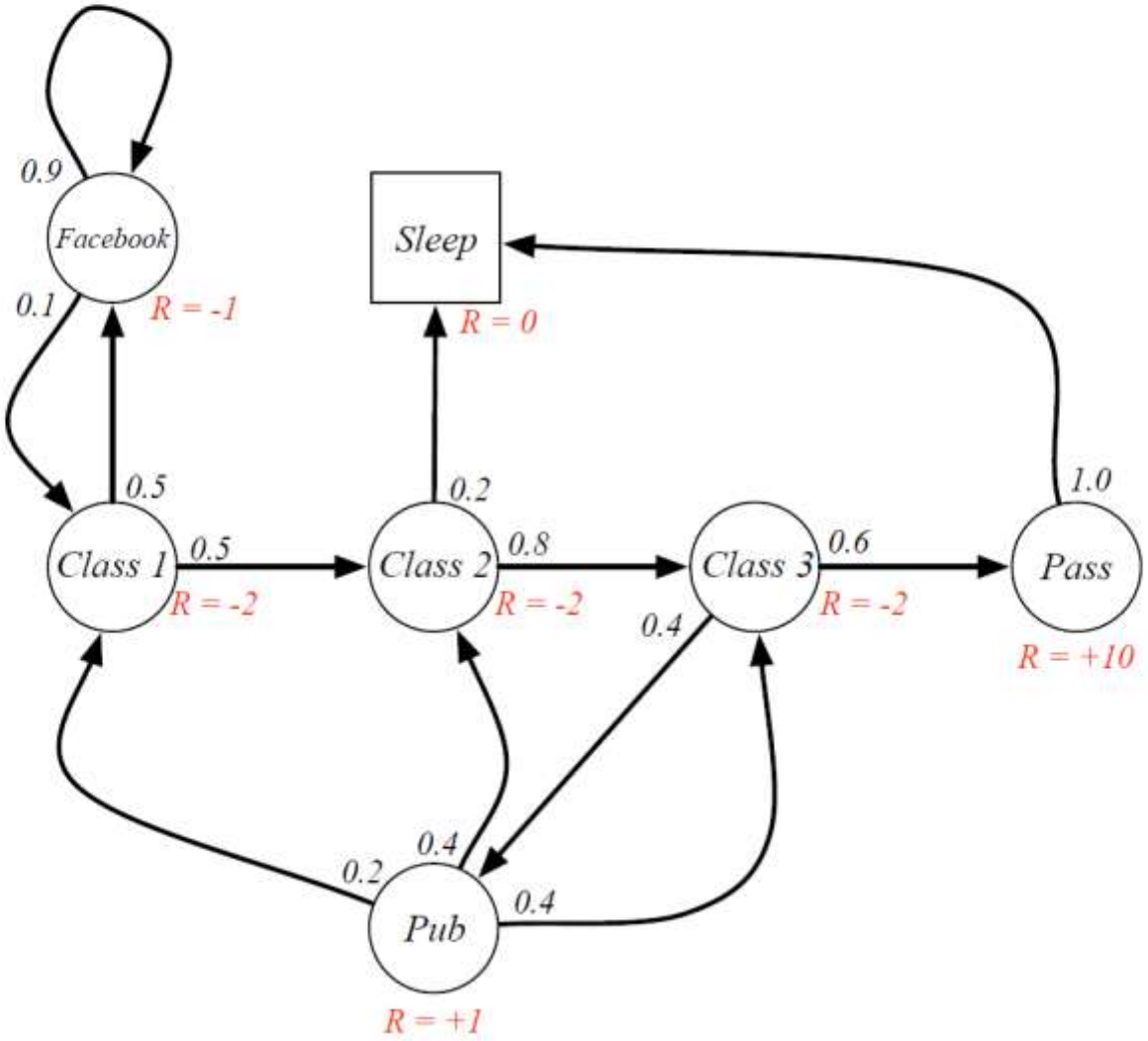
Markov Process



Markov Reward Process



Example: Student MRP



Return

Definition

The *return* G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The *discount* $\gamma \in [0, 1]$ is the present value of future rewards
- The value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$.
- This values immediate reward above delayed reward.
  - γ close to 0 leads to "myopic" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

Why discount?

Most Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behaviour shows preference for immediate reward
- It is sometimes possible to use *undiscounted* Markov reward processes (i.e. $\gamma = 1$), e.g. if all sequences terminate.

Value Function

The value function $v(s)$ gives the long-term value of state s

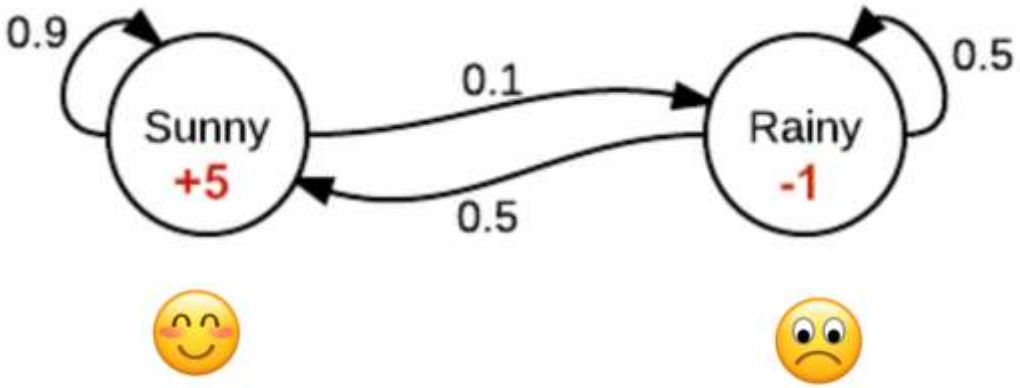
Definition

The *state value function* $v(s)$ of an MRP is the expected return starting from state s

$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$

Value Function

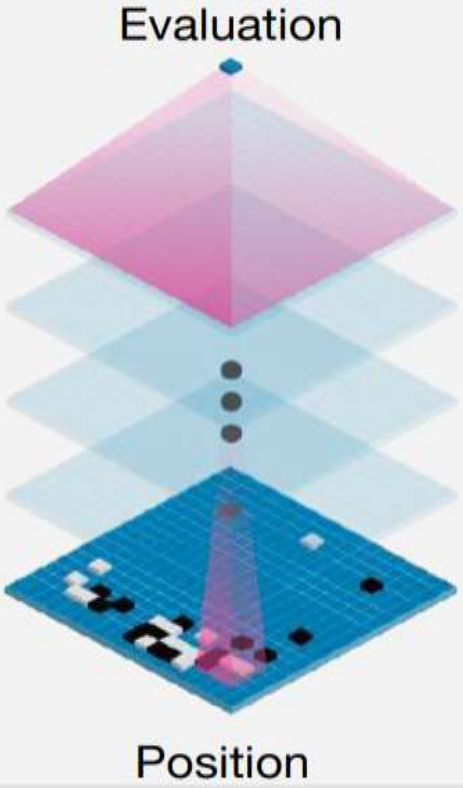
Markov Reward Process

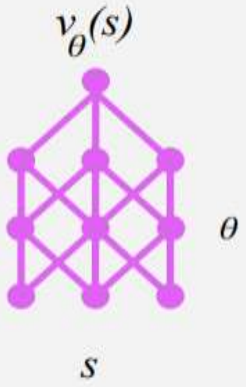


V(Sunny) > V(Rainy)

Value Function

Value network





AlphaGo

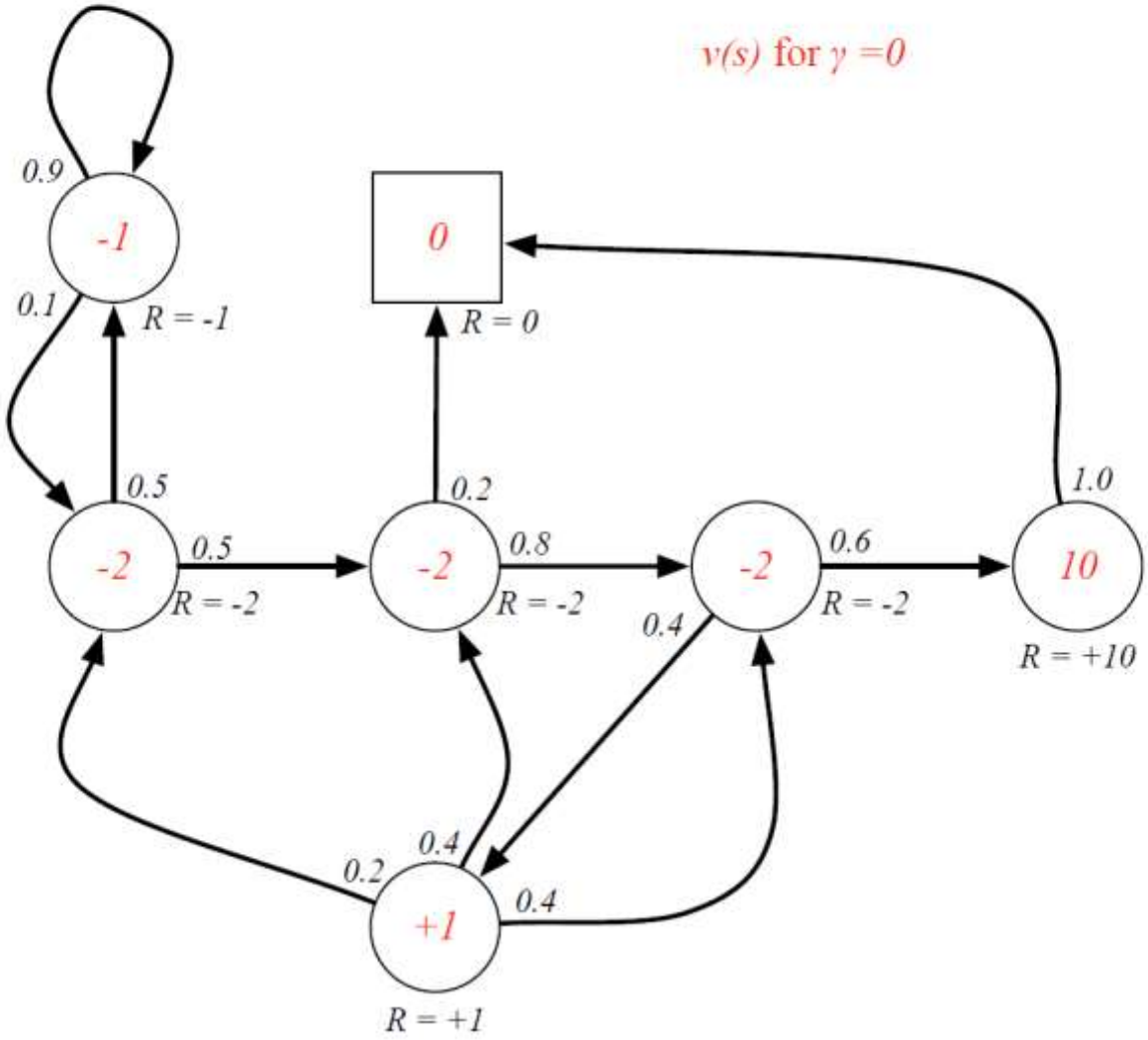
Example: Student MRP Returns

Sample returns for Student MRP:
Starting from $S_1 = \text{C1}$ with $\gamma = \frac{1}{2}$

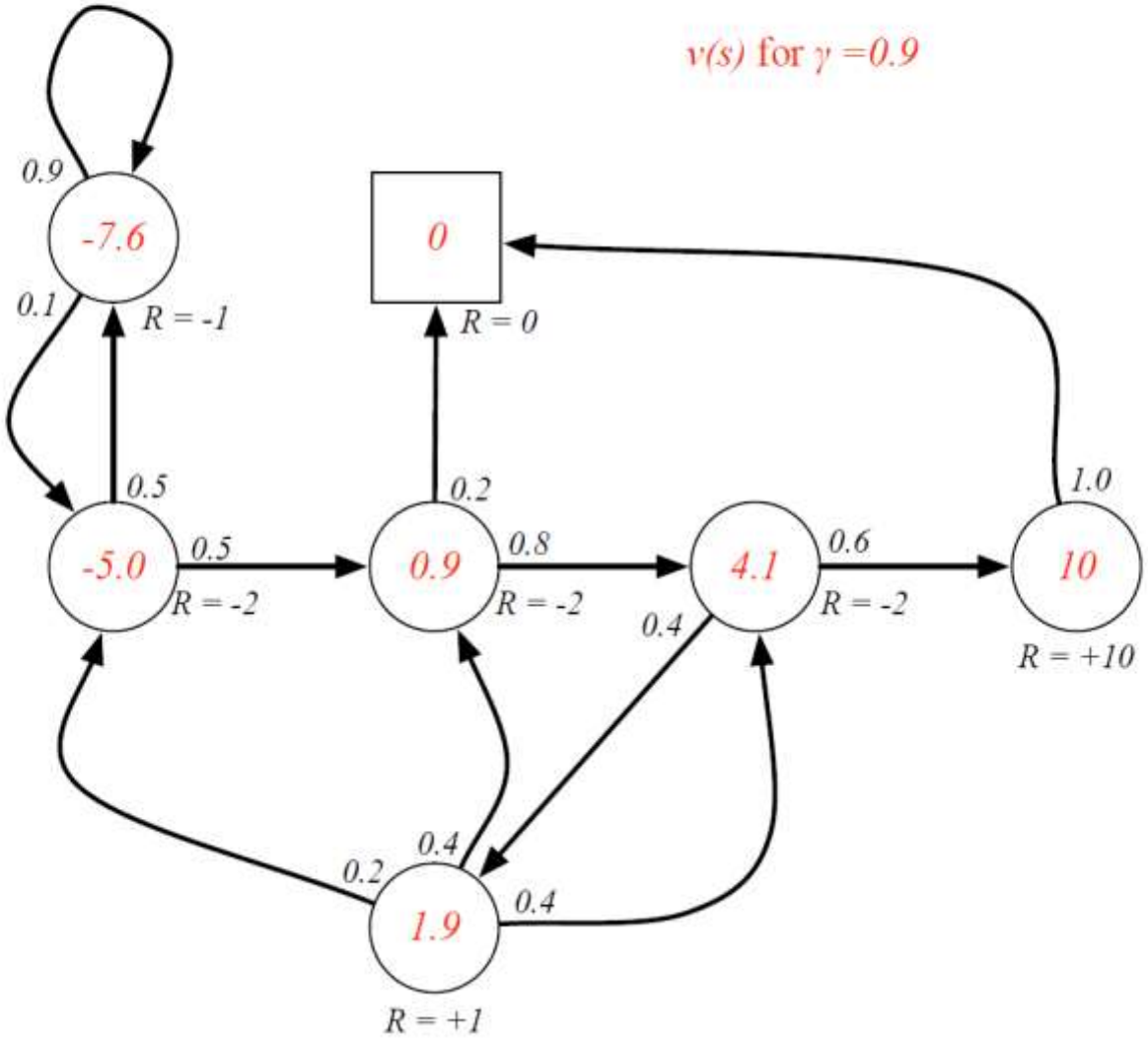
$$G_1 = R_2 + \gamma R_3 + ... + \gamma^{T-2} R_T$$

C1 C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8}$	$=$	-2.25
C1 FB FB C1 C2 Sleep	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16}$	$=$	-3.125
C1 C2 C3 Pub C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \dots$	$=$	-3.41
C1 FB FB C1 C2 C3 Pub C1 ... FB FB FB C1 C2 C3 Pub C2 Sleep	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \dots$	$=$	-3.20

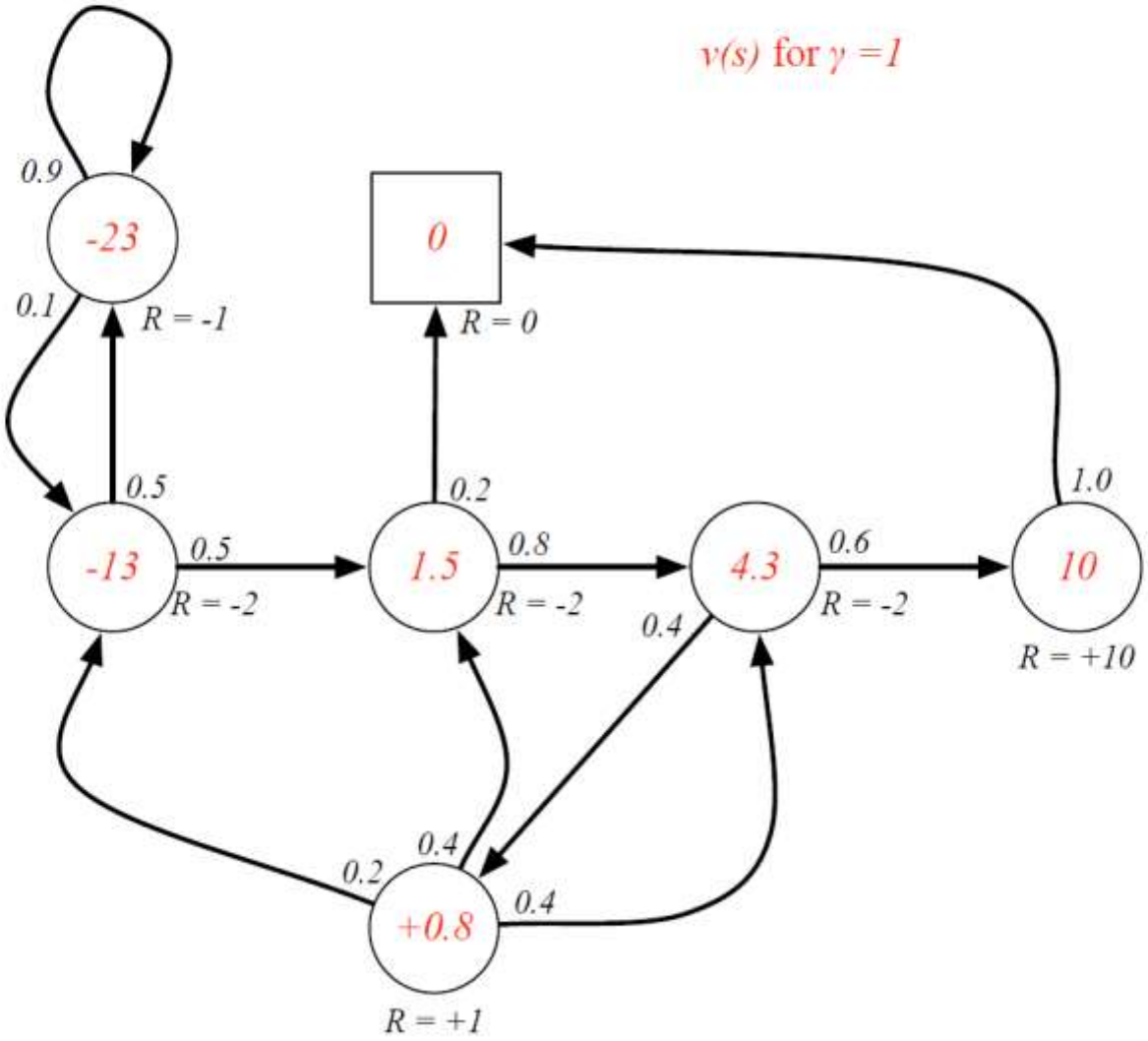
Example: State-Value Function for Student MRP (1)



Example: State-Value Function for Student MRP (2)



Example: State-Value Function for Student MRP (3)



Bellman Equation for MRPs

The value function can be decomposed into two parts:

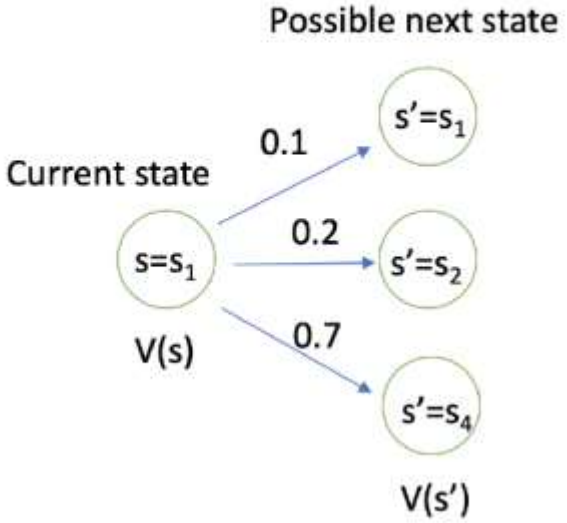
- immediate reward R_{t+1}
- discounted value of successor state $\gamma v(S_{t+1})$

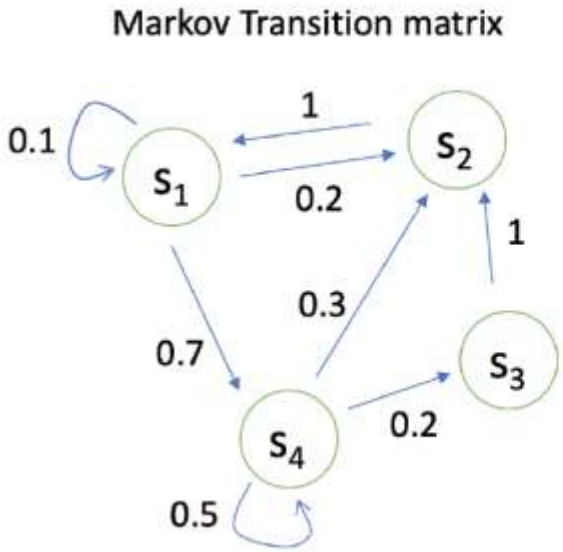
$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma\left(R_{t+2} + \gamma R_{t+3} + \ldots\right) \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]
\end{aligned}
$$

Bellman Equation for MRPs (2)

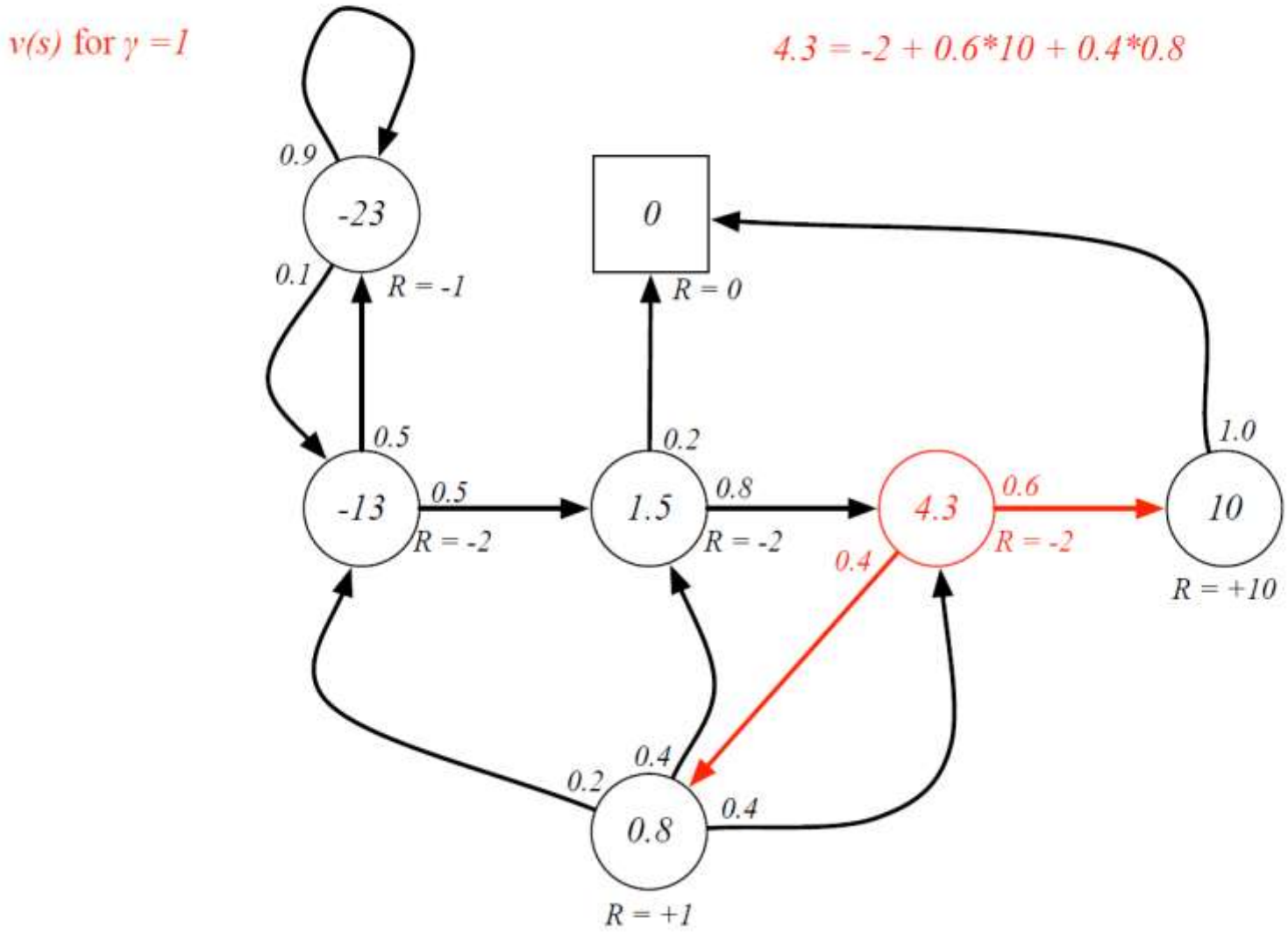
Bellman equation describes the iterative relations of states

$$V(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s) V(s')$$





Example: Bellman Equation for Student MRP



Bellman Equation in Matrix Form

The Bellman equation can be expressed concisely using matrices,

$$v = \mathcal{R} + \gamma \mathcal{P} v$$

where v is a column vector with one entry per state

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{11} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Solving the Bellman Equation

- The Bellman equation is a linear equation
- It can be solved directly:

$$
\begin{aligned}
v &= \mathcal{R} + \gamma \mathcal{P} v \\
(I - \gamma \mathcal{P})\, v &= \mathcal{R} \\
v &= (I - \gamma \mathcal{P})^{-1}\, \mathcal{R}
\end{aligned}
$$

- Computational complexity is $O(n^3)$ for n states
- Direct solution only possible for small MRPs

Solving the Bellman Equation

- There are many iterative methods for large MRPs, e.g.
  - Dynamic programming
  - Monte-Carlo evaluation
  - Temporal-Difference learning

Algorithm Iterative algorithm to calculate MRP value function

1: for all states $s \in S, V'(s) \leftarrow 0, V(s) \leftarrow \infty$
2: **while** $||V - V'|| > \epsilon$ **do**
3: $V \leftarrow V'$
4: For all states $s \in S, V'(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s)V(s')$
5: **end while**
6: return $V'(s)$ for all $s \in S$

Solving the Bellman Equation

- There are many iterative methods for large MRPs, e.g.
  - Dynamic programming
  - Monte-Carlo evaluation
  - Temporal-Difference learning

Algorithm Iterative algorithm to calculate MRP value function

1: for all states $s \in S$, $V'(s) \leftarrow 0$, $V(s) \leftarrow \infty$
2: **while** $||V - V'|| > \epsilon$ **do**
3: $V \leftarrow V'$
4: For all states $s \in S$, $V'(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s) V(s')$
5: **end while**
6: return $V'(s)$ for all $s \in S$

Stop if values change very small

Store current values

Bellman Equation

Updated values

Current values

Markov Decision Process

A Markov decision process (MDP) is a Markov reward process with decisions. It is an *environment* in which all states are Markov.

Definition

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}^{a}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s, A_t = a\right]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}^{a}_{s} = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$
- γ is a discount factor $\gamma \in [0, 1]$.

MRP vs MDP

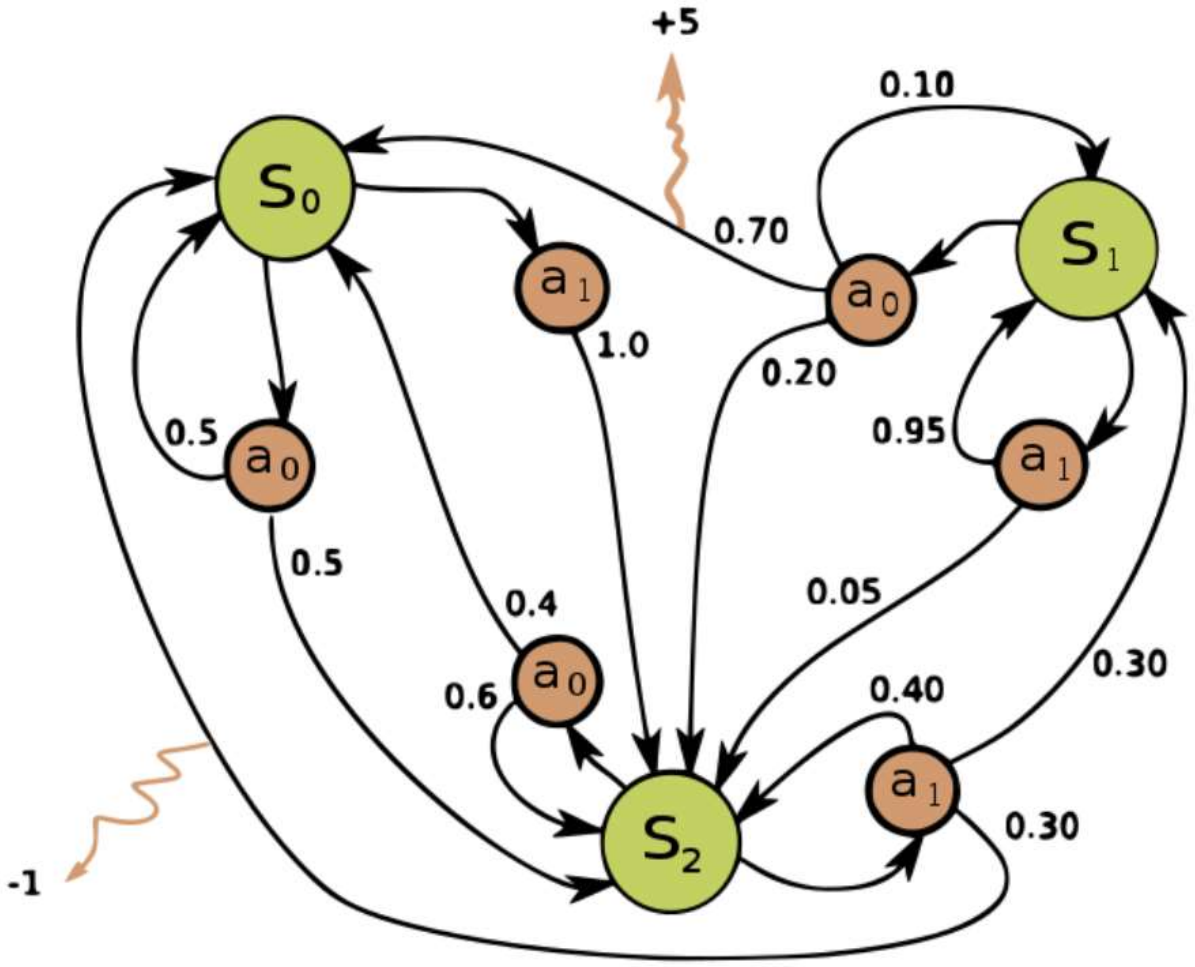


Figure: Markov Chain/MRP: Go with river stream

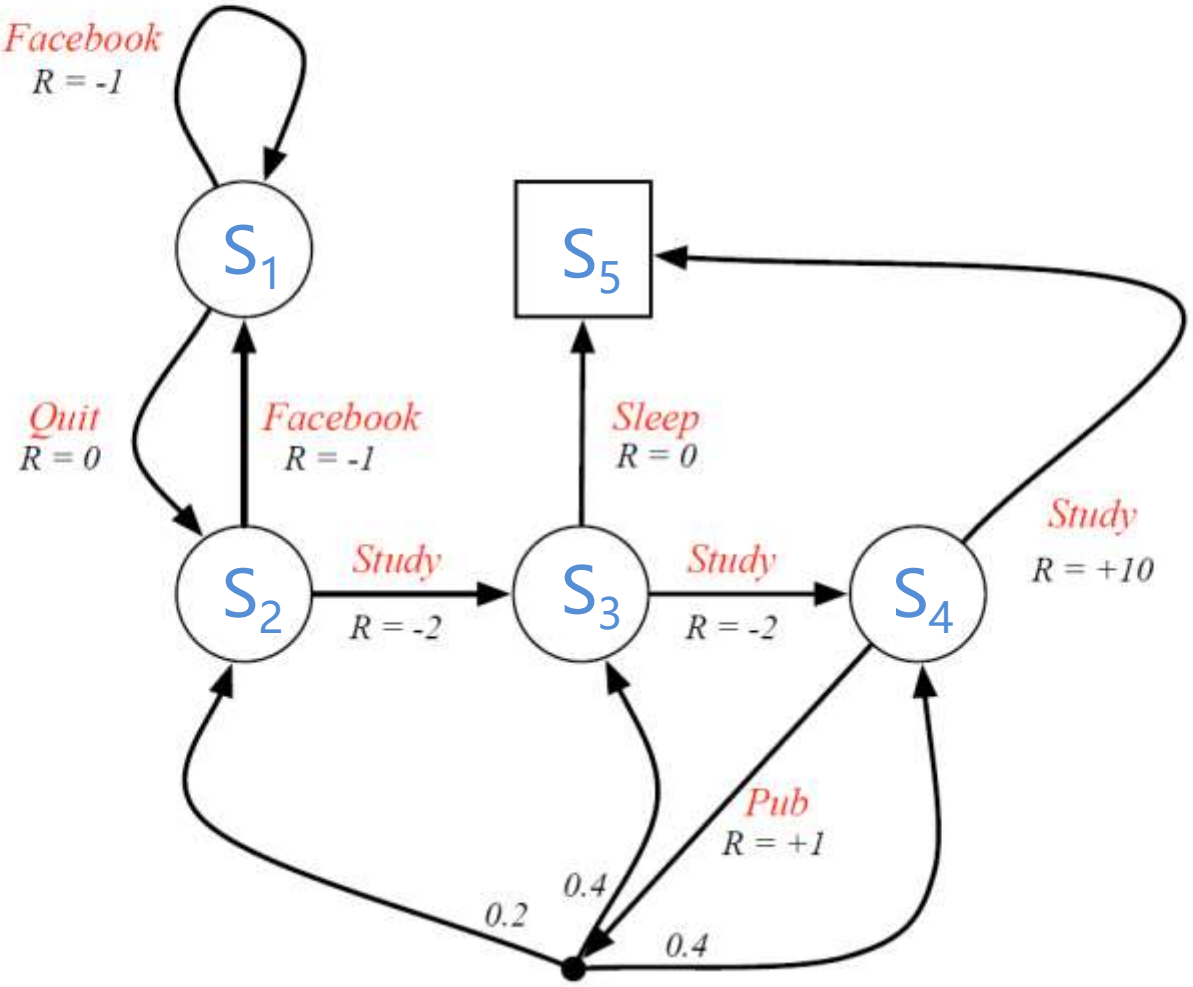


Figure: MDP: Navigate the boat

MDP Example


+5
0.10
s_0
0.70
s_1
a_1
a_0
1.0
0.20
0.5
a_0
0.95
a_1
0.5
0.05
0.4
0.6
a_0
0.40
0.30
a_1
s_2
0.30
-1

Example: Student MDP



Policies (1)

Definition

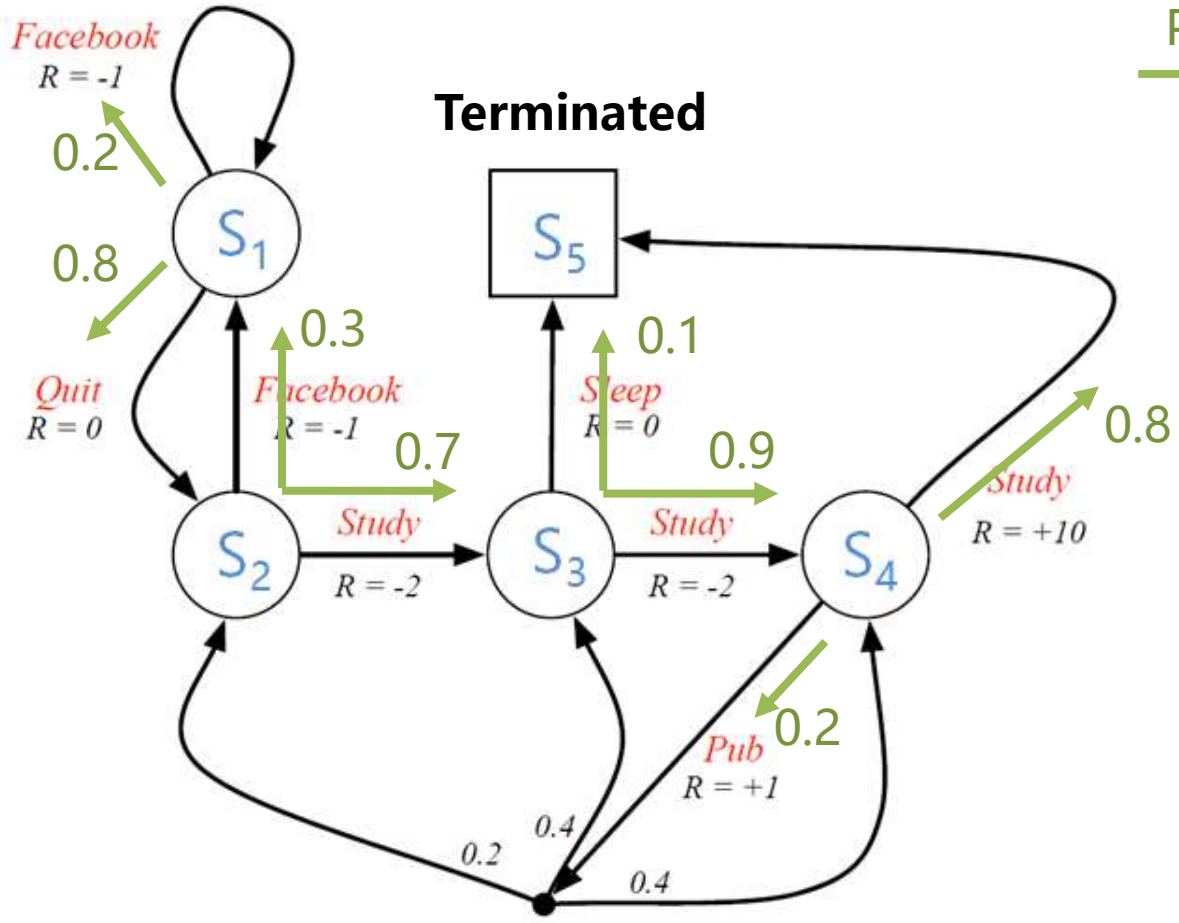
A *policy* π is a distribution over actions given states,

$$\pi(a|s) = \mathbb{P}\left[A_t = a \mid S_t = s\right]$$

- A policy fully defines the behaviour of an agent
- MDP policies depend on the current state (not the history)
- i.e. Policies are *stationary* (time-independent),
  $A_t \sim \pi(\cdot|S_t), \forall t > 0$

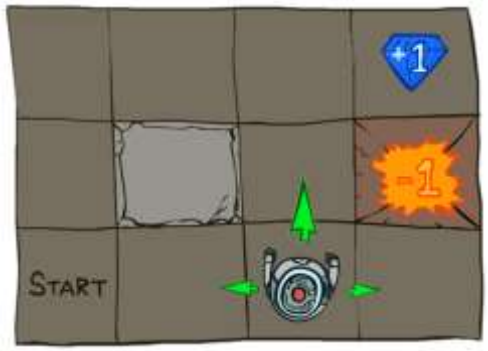
MDP & Policy

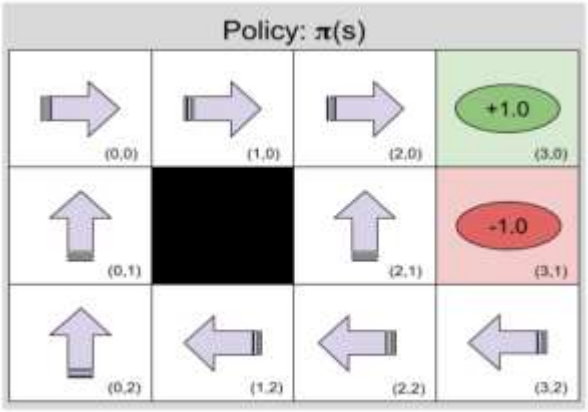
Example: Student MDP



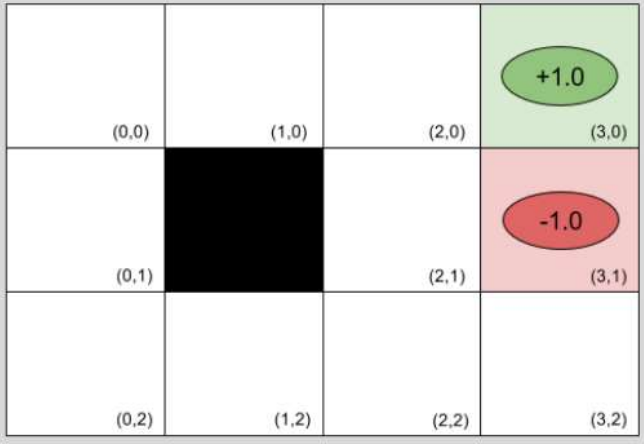
MDP & Policy

Robot Navigation

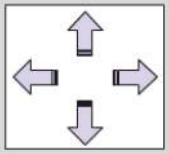




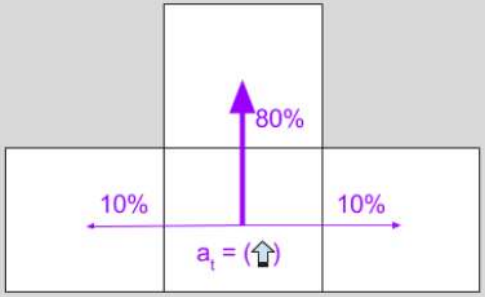
States S and R(s)



Actions A



Transition Model: $P_t(s'_{t+1} \mid a_t, s_t)$



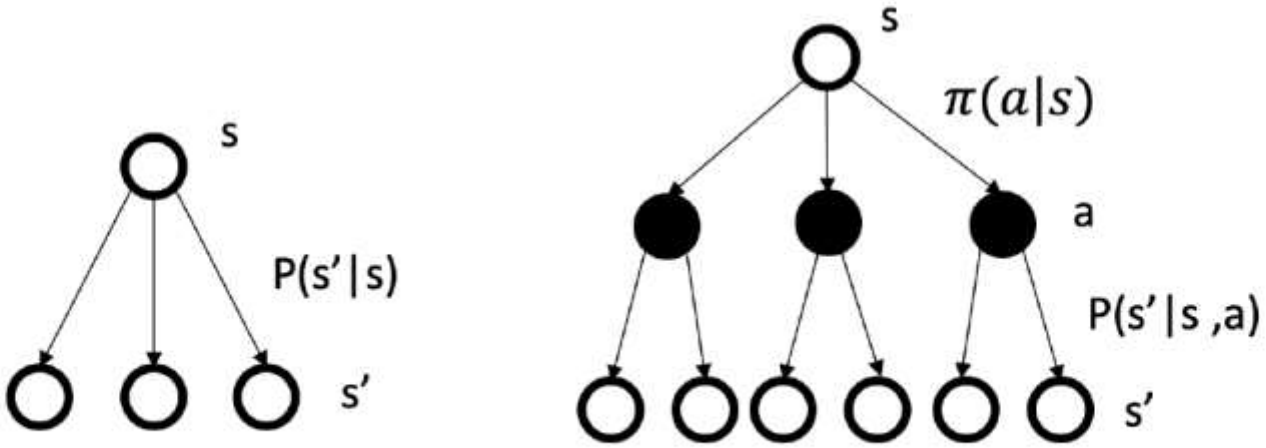
Policies (2)

- Given an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and a policy π
- The state sequence $S_1, S_2, ...$ is a Markov process $\langle \mathcal{S}, \mathcal{P}^\pi \rangle$
- The state and reward sequence $S_1, R_2, S_2, ...$ is a Markov reward process $\langle \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}^\pi, \gamma \rangle$
- where

$$\mathcal{P}^\pi_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}^a_{ss'}$$

$$\mathcal{R}^\pi_s = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}^a_s$$

Comparison of MP/MRP and MDP



Value Function

Definition

The *state-value function* $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π

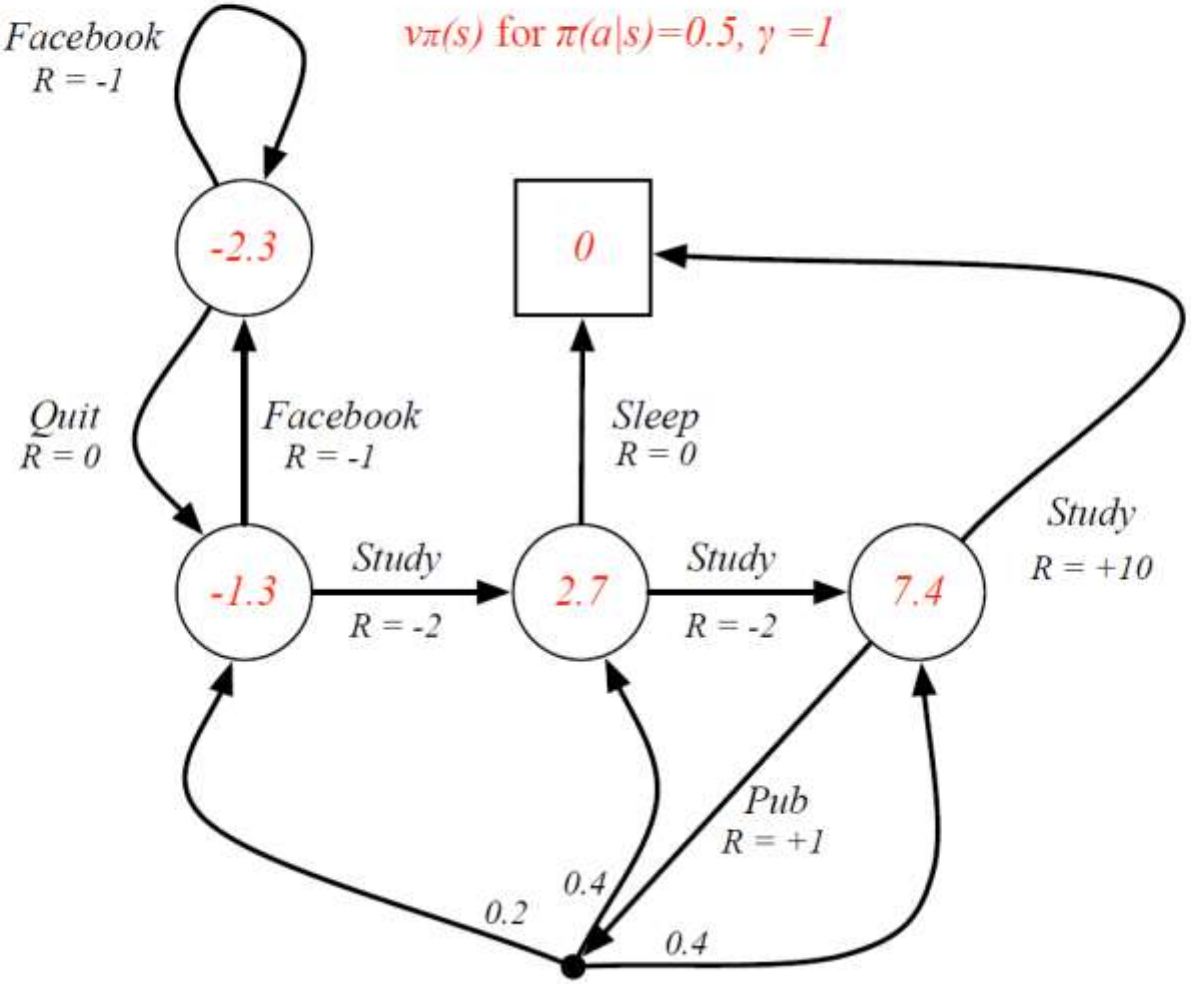
$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

Definition

The *action-value function* $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$$

Example: State-Value Function for Student MDP



Bellman Expectation Equation

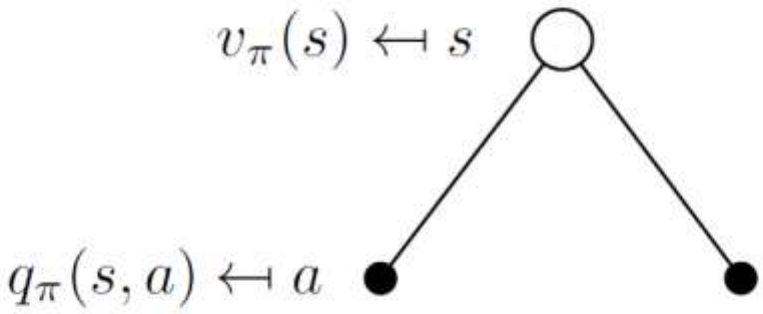
The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right]$$

The action-value function can similarly be decomposed,

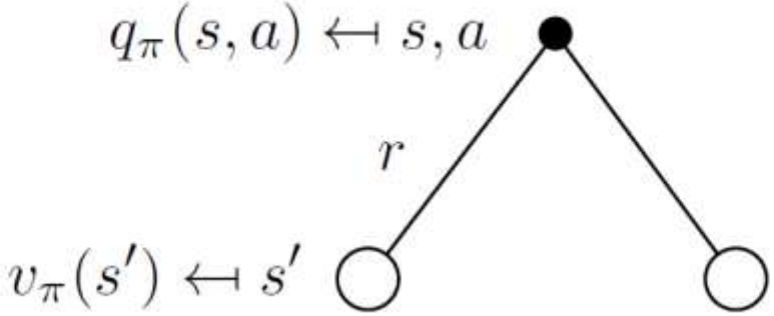
$$q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right]$$

Bellman Expectation Equation for V^π



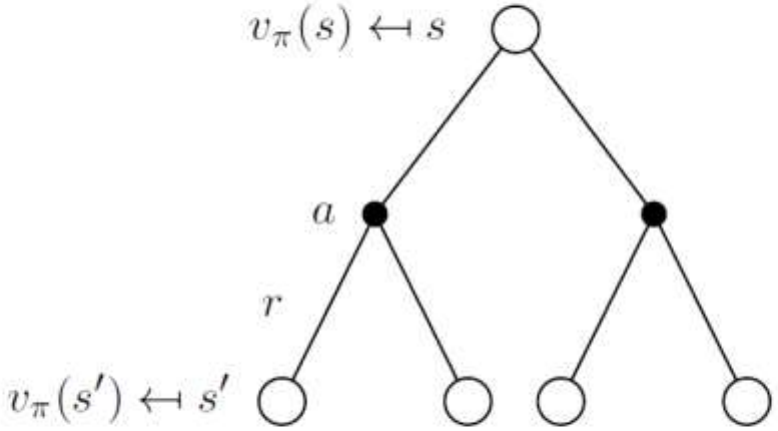
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s,a)$$

Bellman Expectation Equation for Q^π



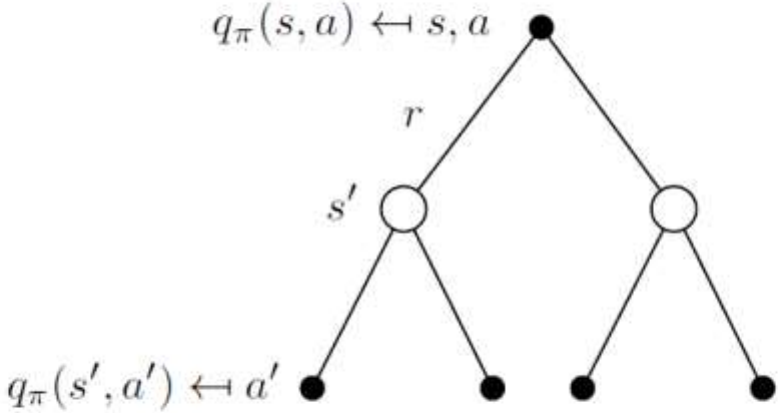
$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

Bellman Expectation Equation for v_π (2)



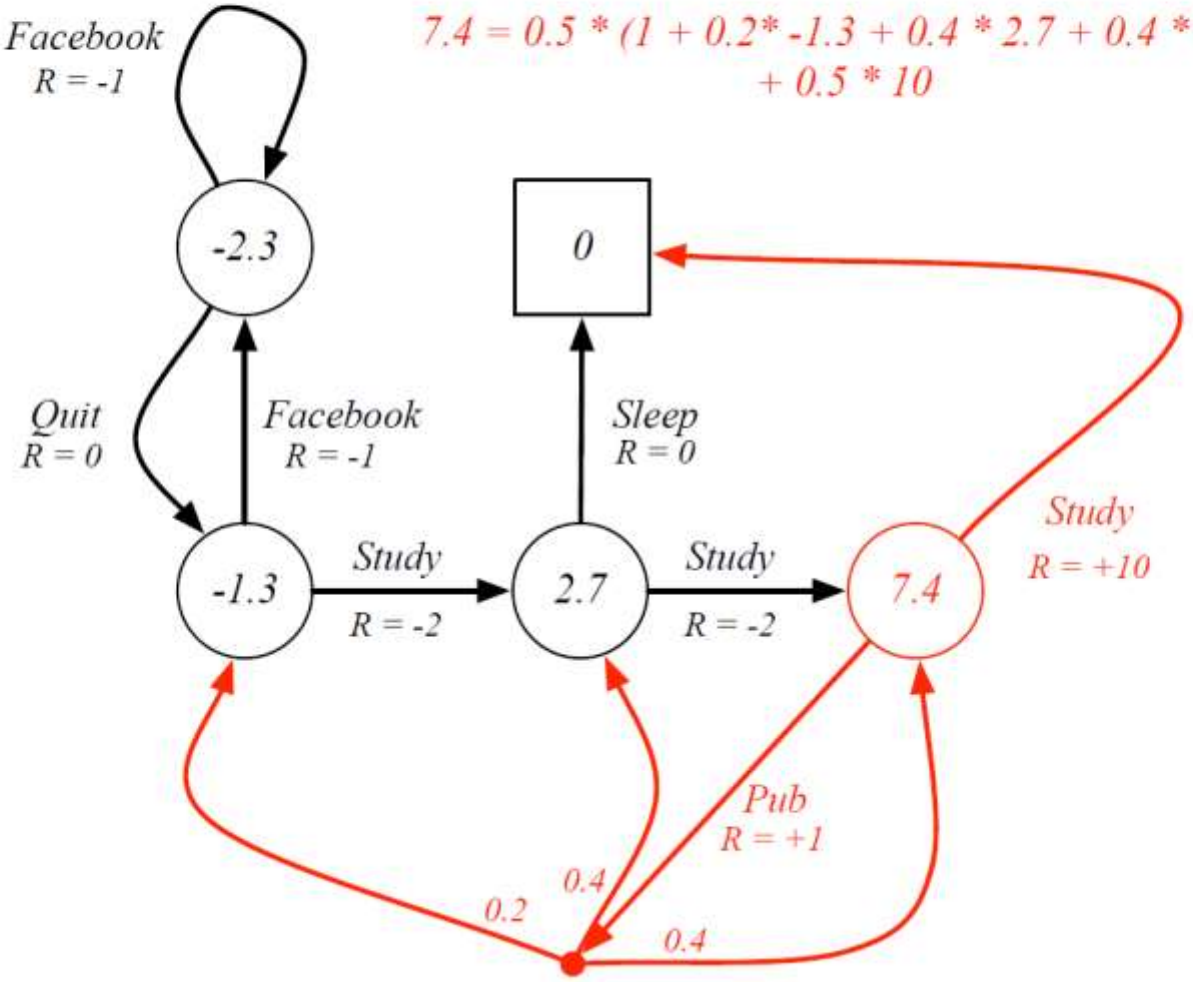
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$

Bellman Expectation Equation for q_π (2)



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a')$$

Example: Bellman Expectation Equation in Student MDP



Optimal Policies for Markov Decision Process

Optimal Value Function

Definition

The *optimal state-value function* $v_*(s)$ is the maximum value function over all policies

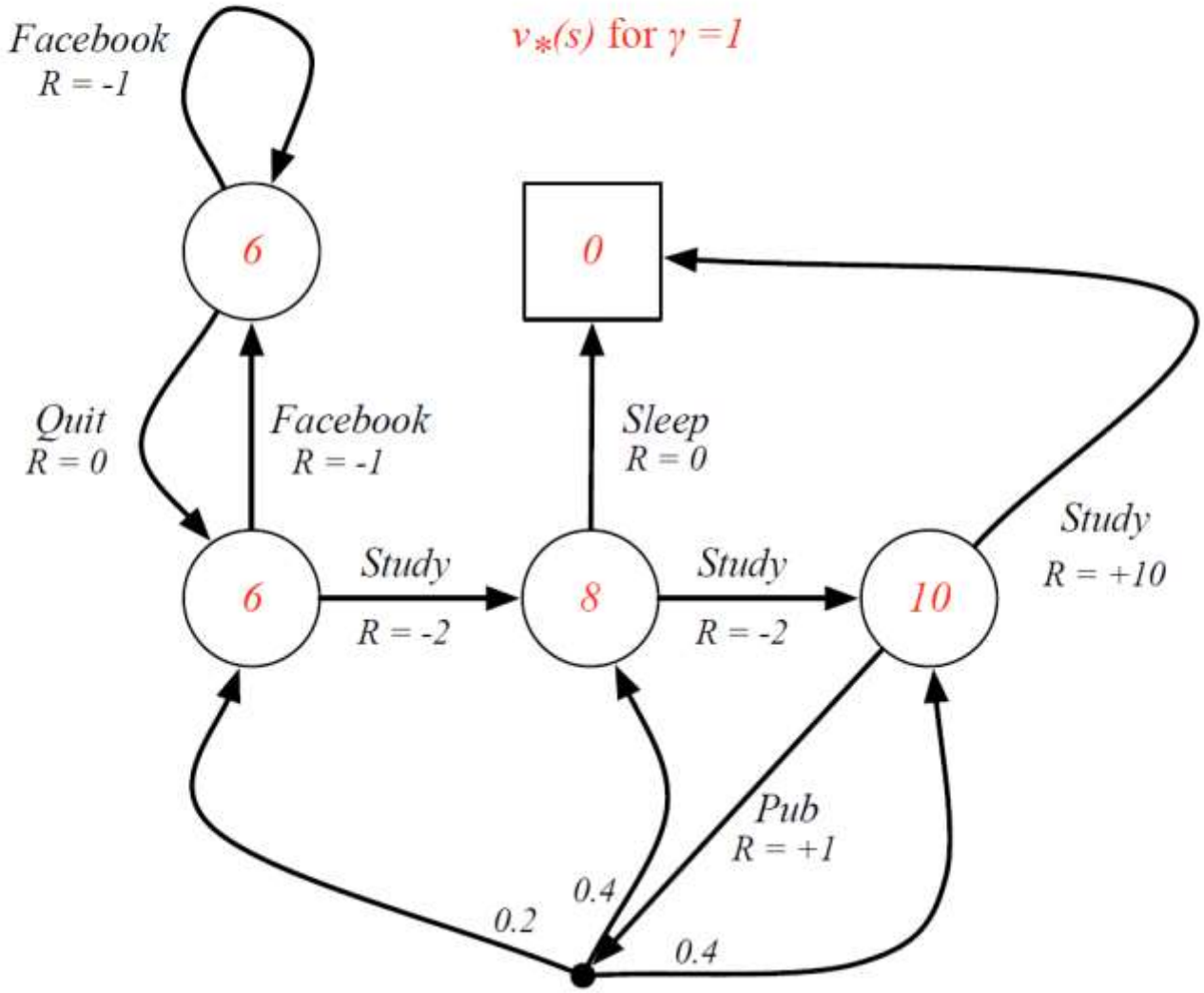
$$v_*(s) = \max_\pi v_\pi(s)$$

The *optimal action-value function* $q_*(s, a)$ is the maximum action-value function over all policies

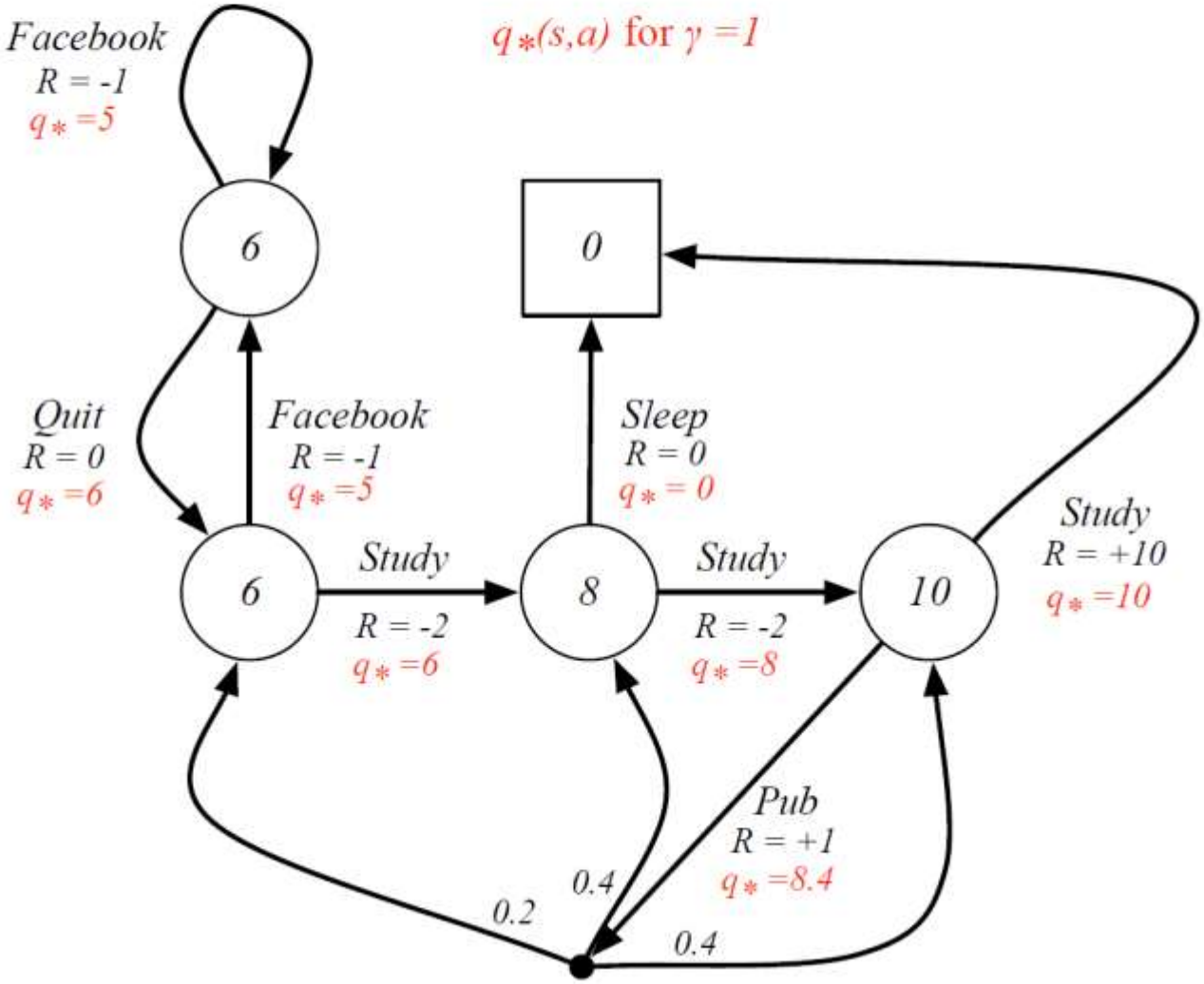
$$q_*(s, a) = \max_\pi q_\pi(s, a)$$

- The optimal value function specifies the best possible performance in the MDP.
- An MDP is "solved" when we know the optimal value.

Example: Optimal Value Function for Student MDP



Example: Optimal Action-Value Function for Student MDP



Optimal Policy

Define a partial ordering over policies

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s$$

Theorem

For any Markov Decision Process

- *There exists an optimal policy π_* that is better than or equal to all other policies, $\pi_* \geq \pi, \forall \pi$*
- *All optimal policies achieve the optimal value function, $v_{\pi_*}(s) = v_*(s)$*
- *All optimal policies achieve the optimal action-value function, $q_{\pi_*}(s, a) = q_*(s, a)$*

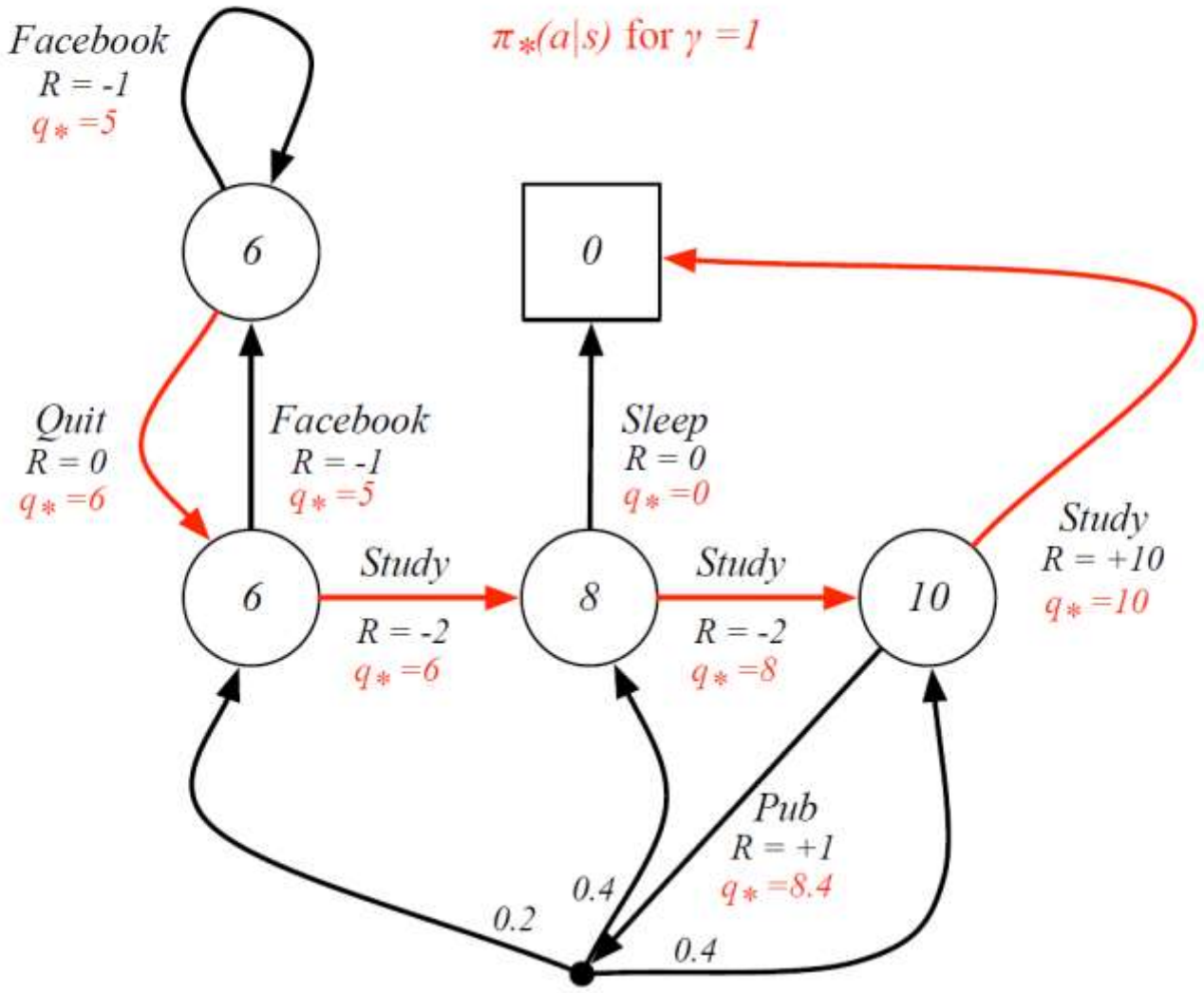
Finding an Optimal Policy

An optimal policy can be found by maximising over $q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s, a) \\ 0 & \textit{otherwise} \end{cases}$$

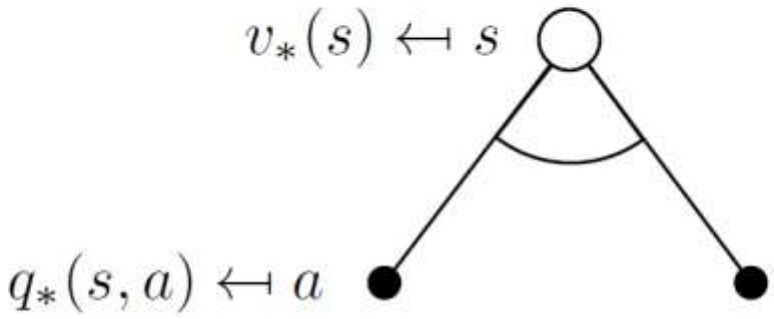
- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, we immediately have the optimal policy

Example: Optimal Policy for Student MDP



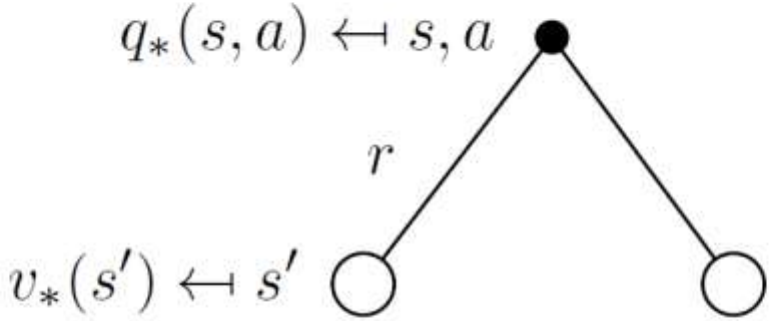
Bellman Optimality Equation for v_*

The optimal value functions are recursively related by the Bellman optimality equations:



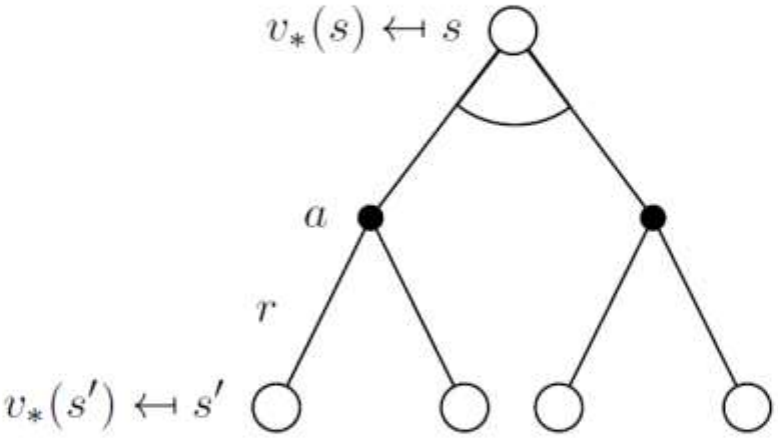
$$v_*(s) = \max_a q_*(s,a)$$

Bellman Optimality Equation for Q^*



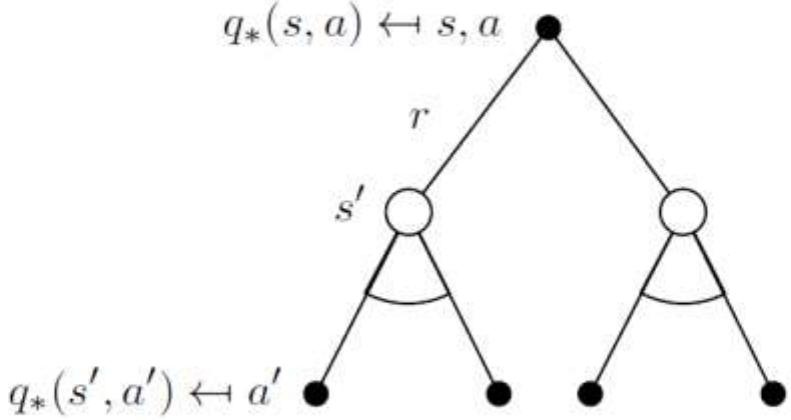
$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Bellman Optimality Equation for V^* (2)



$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Bellman Optimality Equation for Q^* (2)



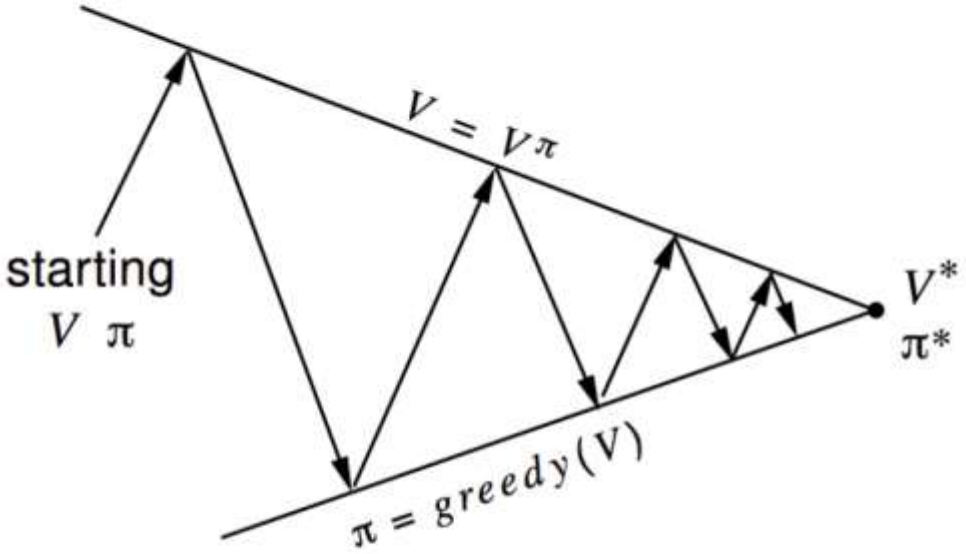
$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a')$$

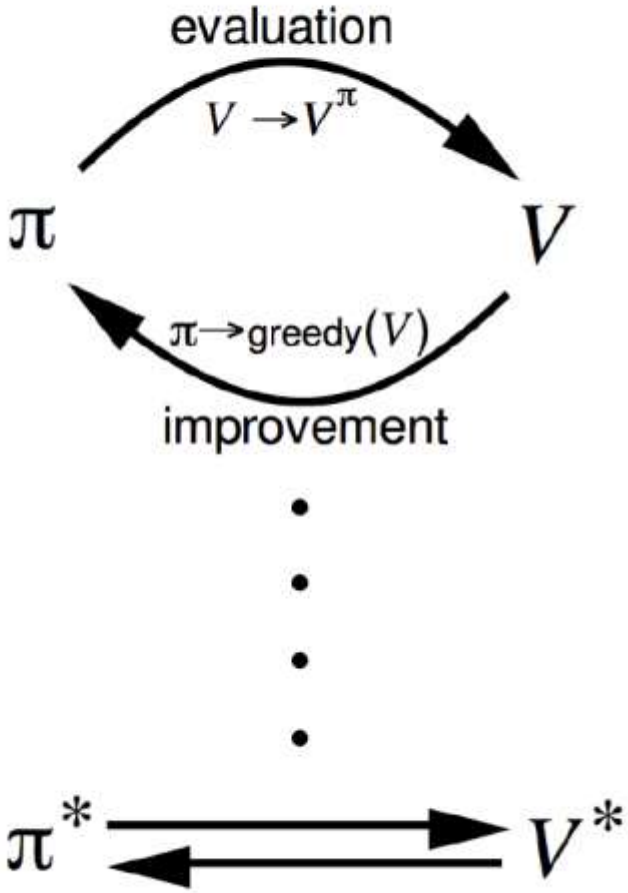
Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear
- No closed form solution (in general)
- Many iterative solution methods
  - Value Iteration
  - Policy Iteration
  - Q-learning
  - Sarsa

Policy Iteration for Optimal Policies

Policy Iteration





Policy evaluation Estimate v_π

Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$

Greedy policy improvement

Policy Evaluation on MDP

The state and reward sequence $S_1, R_2, S_2, \ldots$ is a Markov reward process

$$\mathcal{P}_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}^a_{ss'}$$

$$\mathcal{R}_s = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}^a_s$$

Transformed as a MRP for a fixed policy

Directly Solve Bellman Equation

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \cdots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{11} & \cdots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Small-scale Problem

Iteratively Solve Bellman Equation

Algorithm Iterative algorithm to calculate MRP value function

1: for all states $s \in S, V'(s) \leftarrow 0, V(s) \leftarrow \infty$
2: **while** $||V - V'|| > \epsilon$ **do**
3: $V \leftarrow V'$
4: For all states $s \in S, V'(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s) V(s')$
5: **end while**
6: return $V'(s)$ for all $s \in S$

Large-scale Problem

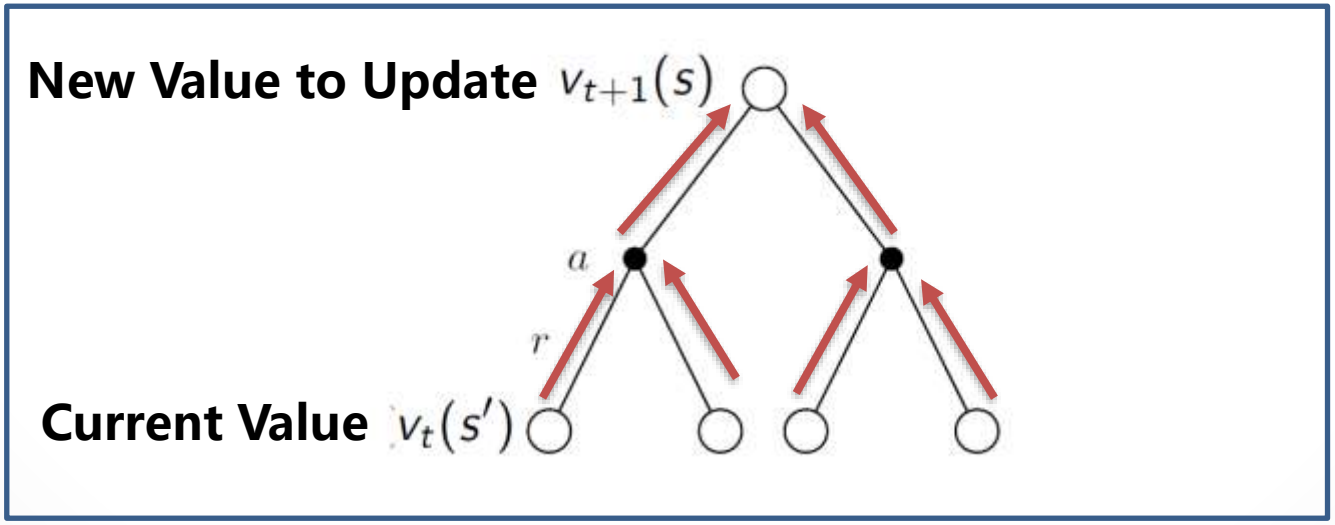
Policy evaluation: Iteration on Bellman expectation backup

Bellman expectation backup for a particular policy

$$v_{t+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s)(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) v_t(s'))$$

Or if in the form of MRP $< \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}, \gamma >$

$$v_{t+1}(s) = R^\pi(s) + \gamma P^\pi(s'|s) v_t(s')$$



Policy evaluation on MDP

Algorithm

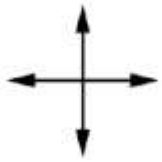
At each iteration t+1

update $v_{t+1}(s)$ from $v_t(s')$ for all states $s \in \mathcal{S}$ where s' is a successor state of s

$$v_{t+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s)(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) v_t(s'))$$

Convergence: $v_1 \rightarrow v_2 \rightarrow ... \rightarrow v^{\pi}$

Evaluating a Random Policy in the Small Gridworld



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$r = -1$
on all transitions

- Undiscounted episodic MDP ($\gamma = 1$)
- Nonterminal states $1, \ldots, 14$
- One terminal state (shown twice as shaded squares)
- Actions leading out of the grid leave state unchanged
- Reward is -1 until the terminal state is reached
- Agent follows uniform random policy

$$\pi(n|\cdot) = \pi(e|\cdot) = \pi(s|\cdot) = \pi(w|\cdot) = 0.25$$

Iterative Policy Evaluation in Small Gridworld

v_k for the Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

How to Improve a Policy

- Given a policy π
  - Evaluate the policy π

$$v_\pi(s) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + ... | S_t = s\right]$$

  - Improve the policy by acting greedily with respect to v_π

$$\pi' = \text{greedy}(v_\pi)$$

Policy Improvement

1. Compute the state-action value of a policy π:

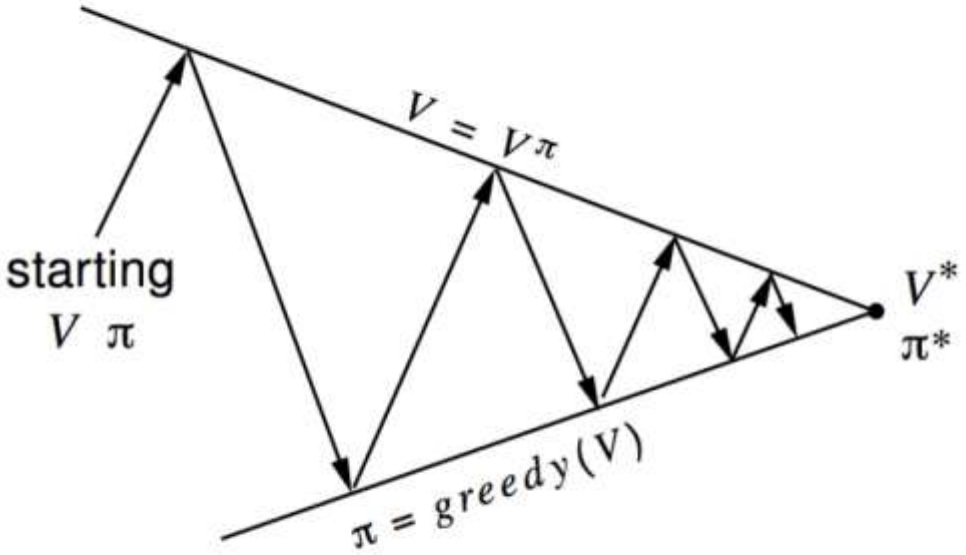
$$q^{\pi_i}(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) v^{\pi_i}(s')$$

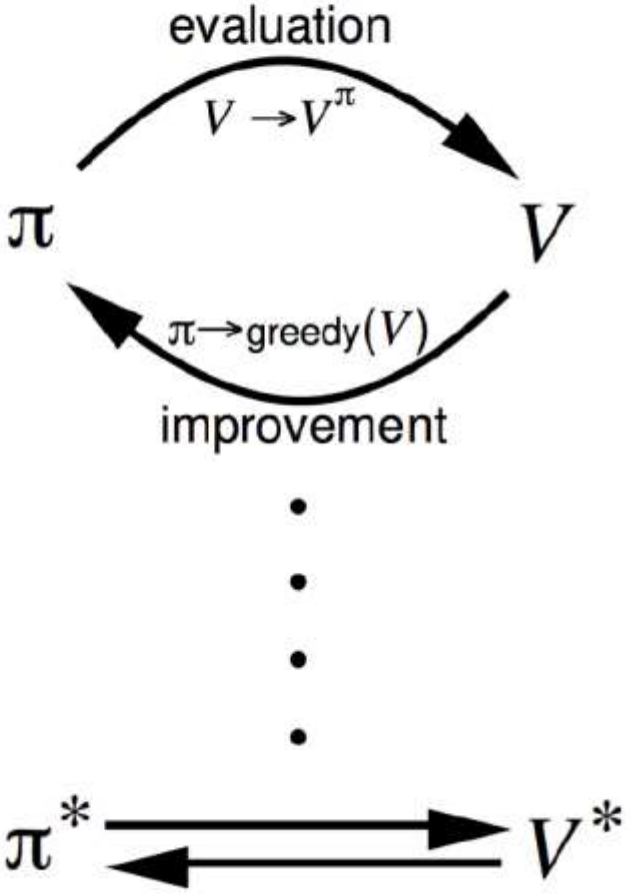
2. Compute new policy π_{i+1} for all $s \in \mathcal{S}$ following

$$\pi_{i+1}(s) = \arg\max_a q^{\pi_i}(s,a)$$



Policy Iteration





Policy evaluation Estimate v_π

Iterative policy evaluation

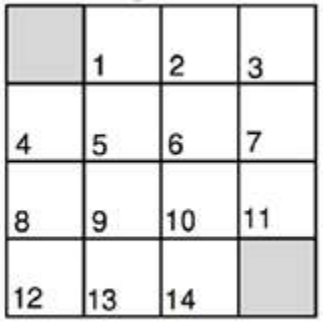
Policy improvement Generate $\pi' \geq \pi$

Greedy policy improvement

Small Gridworld



actions

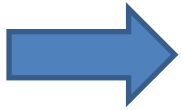


$r = -1$
on all transitions

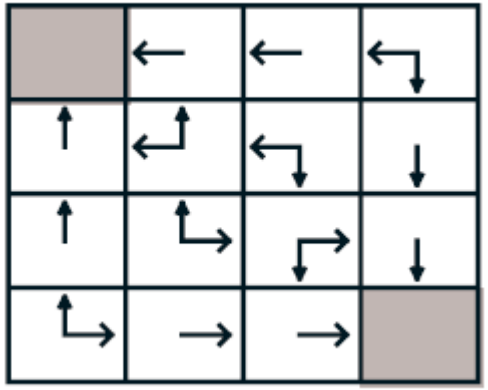
Policy Evaluation of Random Policy

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

Policy Update



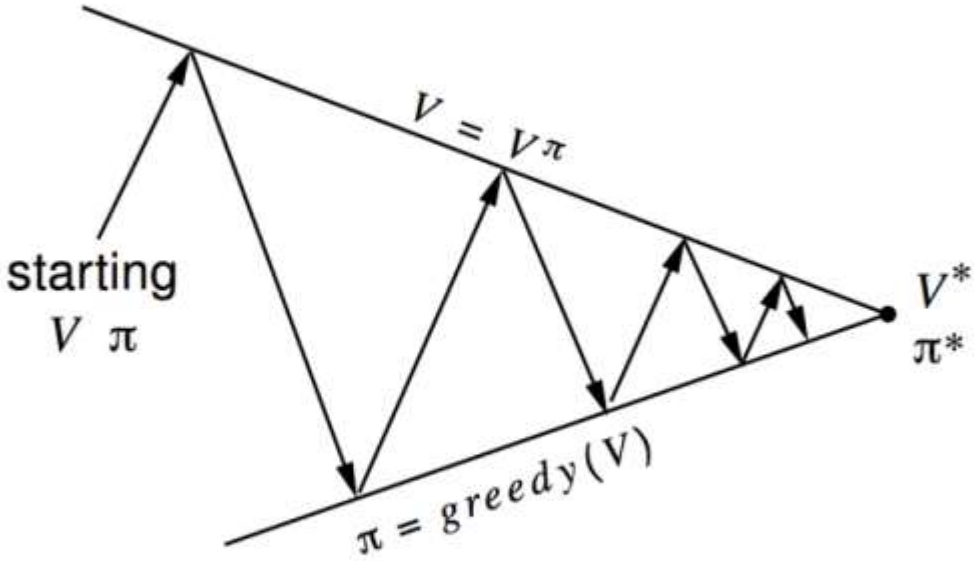
Optimal Policy!

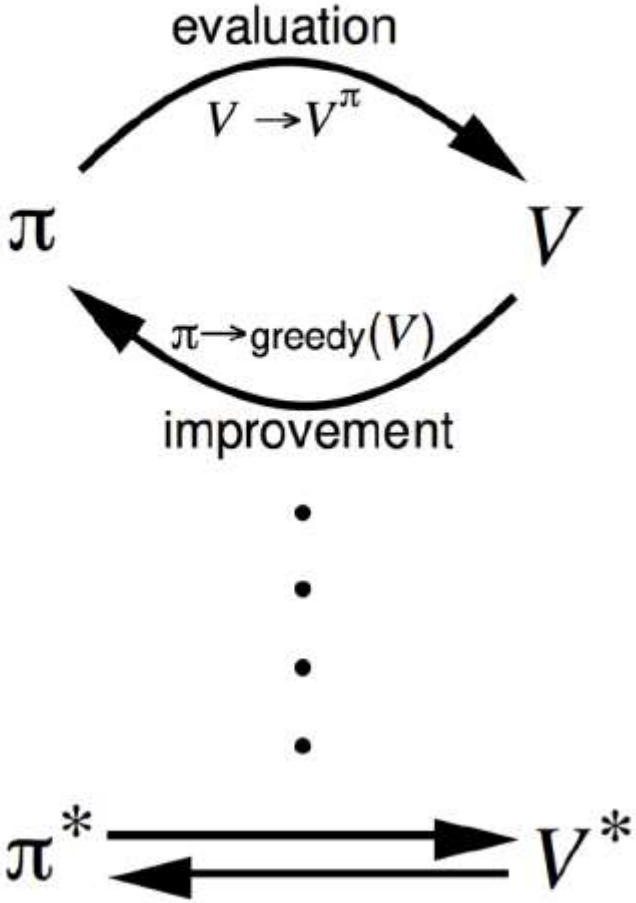


Modified Policy Iteration

- Does policy evaluation need to converge to v_π?
- Or should we introduce a stopping condition
  - e.g. ϵ-convergence of value function
- Or simply stop after k iterations of iterative policy evaluation?

Generalised Policy Iteration





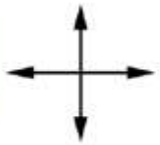
Policy evaluation Estimate v_π

Any policy evaluation algorithm

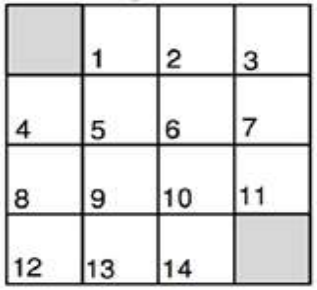
Policy improvement Generate $\pi' \geq \pi$

Any policy improvement algorithm

Small Gridworld



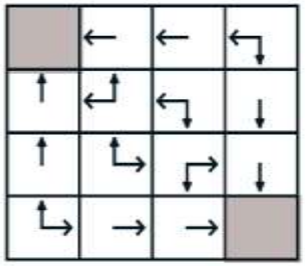
actions



$r = -1$
on all transitions

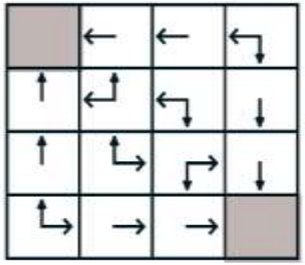
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



$k = 10$

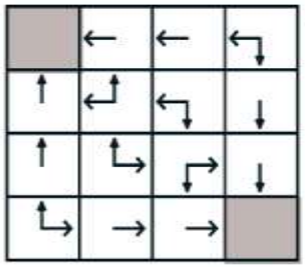
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



optimal policy

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



Value Iteration for Optimal Policies

Bellman Optimality Equation

1. The optimal value functions are reached by the Bellman optimality equations:

$$v^*(s) = \max_a q^*(s,a)$$

$$q^*(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) v^*(s')$$

thus

$$v^*(s) = \max_a R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) v^*(s')$$

$$q^*(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) \max_{a'} q^*(s',a')$$

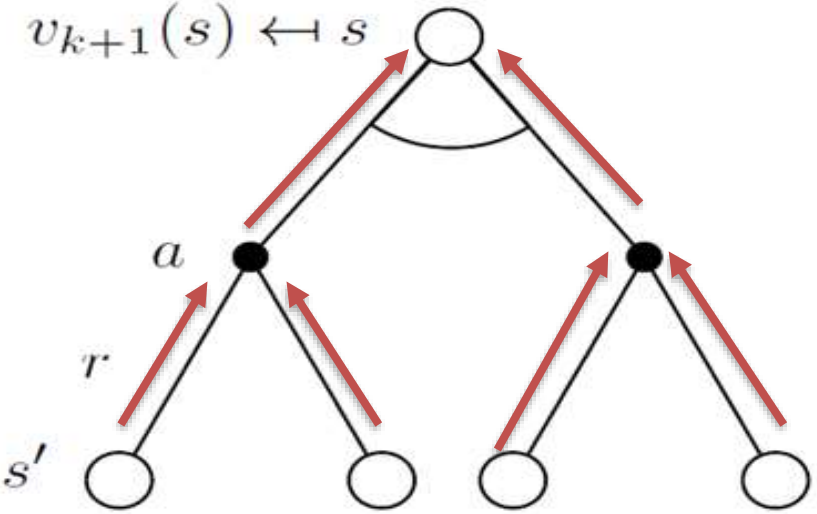
Value Iteration by turning the Bellman Optimality Equation as update rule

1. If we know the solution to subproblem $v^*(s')$, which is optimal.
2. Then the solution for the optimal $v^*(s)$ can be found by iteration over the following Bellman Optimality backup rule,

$$v(s) \leftarrow \max_{a \in \mathcal{A}} \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) v(s') \right)$$

3. The idea of value iteration is to apply these updates iteratively

New Value to Update:



Current Value:

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

Algorithm of Value Iteration

1. Objective: find the optimal policy π
2. Solution: iteration on the Bellman optimality backup
3. Value Iteration algorithm:
   1. initialize $k = 1$ and $v_0(s) = 0$ for all states s
   2. For $k = 1 : H$
      1. for each state s

$$q_{k+1}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) v_k(s')$$

$$v_{k+1}(s) = \max_a q_{k+1}(s, a)$$

      2. $k \leftarrow k + 1$
   3. To retrieve the optimal policy after the value iteration:

$$\pi(s) = \arg\max_a R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) v_{k+1}(s')$$

Algorithm of Value Iteration

1. Objective: find the optimal policy π
2. Solution: iteration on the Bellman optimality backup
3. Value Iteration algorithm:
   1. initialize $k = 1$ and $v_0(s) = 0$ for all states s
   2. For $k = 1 : H$
      1. for each state s

$$q_{k+1}(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) v_k(s')$$

$$v_{k+1}(s) = \max_a q_{k+1}(s,a)$$

      2. $k \leftarrow k + 1$
   3. To retrieve the optimal policy after the value iteration:

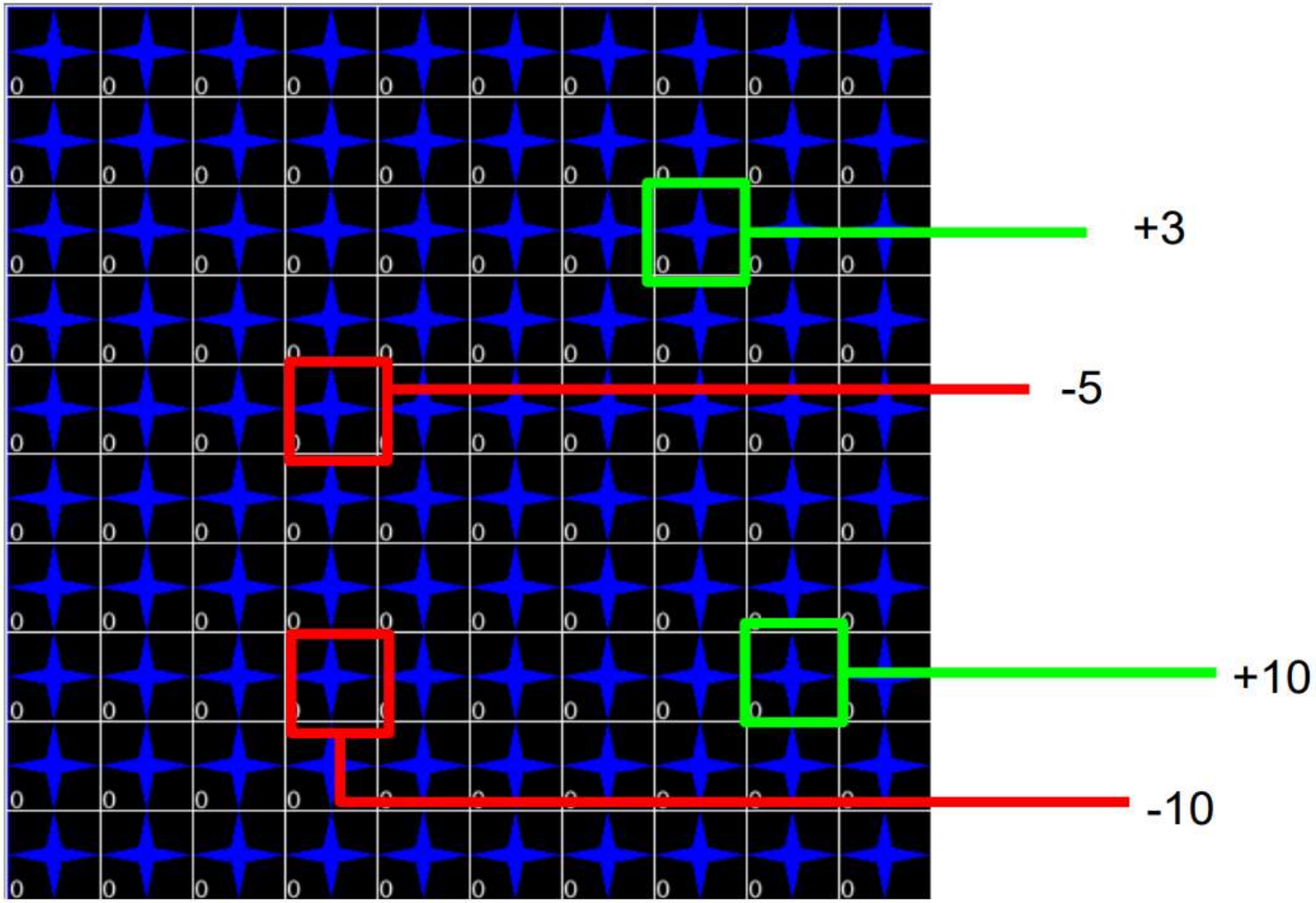
$$\pi(s) = \arg\max_a R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) v_{k+1}(s')$$

$$\pi(s) = \arg\max_a q^{\pi_i}(s,a)$$

Value iteration example: Grid World

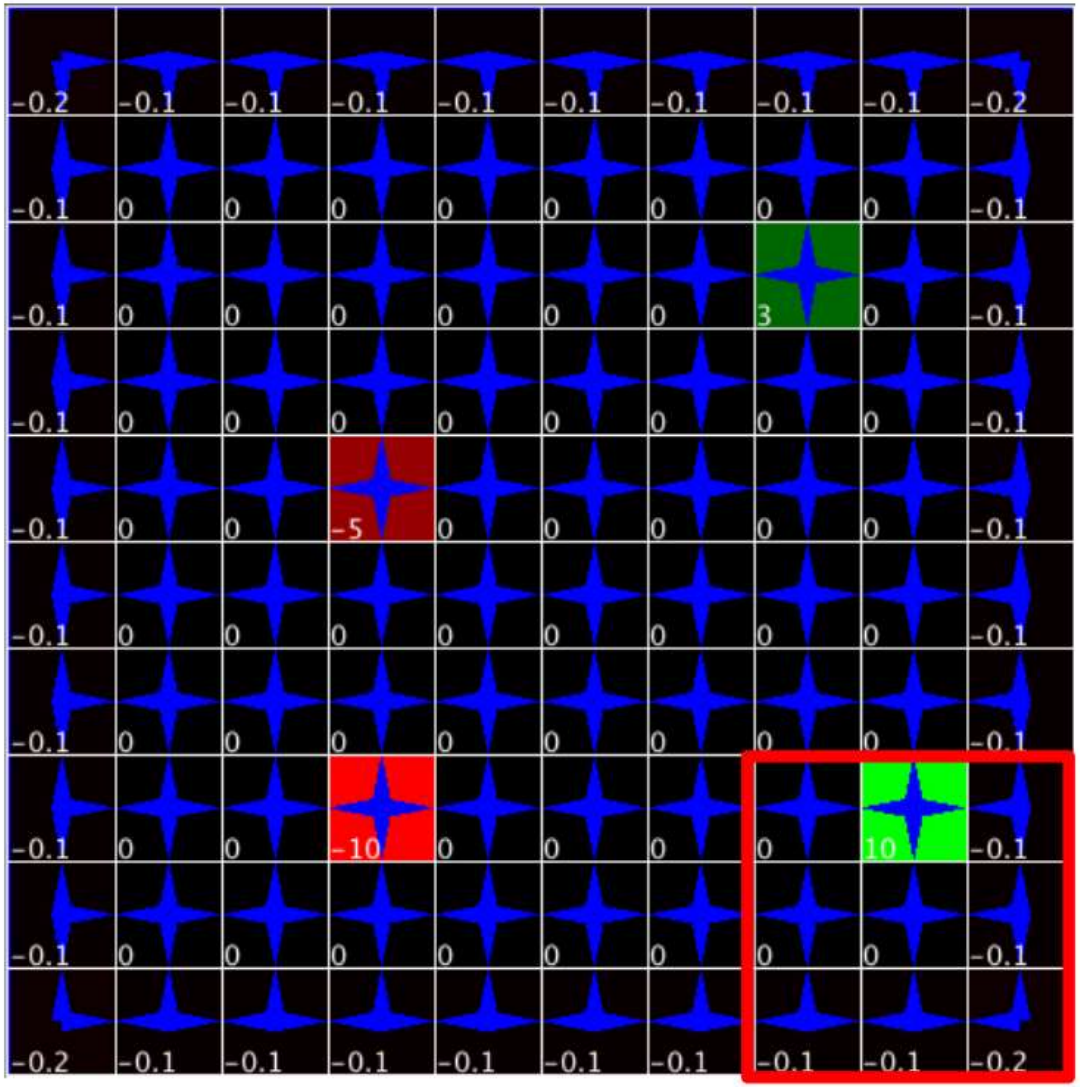
- States: (x,y)-coordinate in a 10 x 10 grid
- Actions: Up, down, left, and right
- Dynamics:
  - Move one-step in direction specified by action with prob 0.7
  - Move one-step in any one of the other directions with prob 0.1
  - Cannot move outside of the grid (i.e. end up in the same state)
  - Agent is flung randomly to corner of grid after entering a Goal or Penalty state
- Rewards:
  - Attempted move outside of grid leads to reward of -1
  - Goal/Penalty states in grid: +10@(9,8), +3@(8,3), -5@(4,5), -10@(4,8)
  - No reward in any other states
- Discount factor: 0.9

Value iteration example: Grid World



10 x 10 Grid World Java Applet:
http://www.cs.ubc.ca/~poole/demos/mdp/vi.html

Value iteration example: Grid World



$V_1(x,y)$ (Iteration 1)

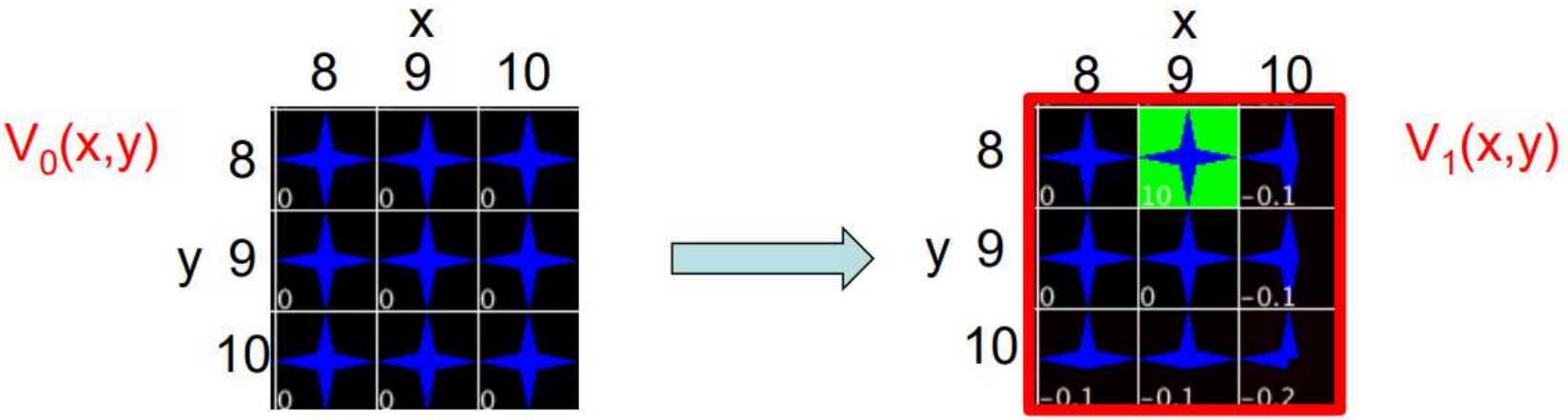
How are these calculated?

Value iteration example: Grid World

$$V_1^*(9,8) = \max_a \left[R(9,8,a) + \gamma \sum_{(x',y')} p(x',y' \mid 9,8,a) V_0^*(x',y') \right]$$

$$= \max_a R(9,8,a)$$

$$= \max_a \{ \underbrace{10}_{\text{up}}, \underbrace{10}_{\text{down}}, \underbrace{10}_{\text{left}}, \underbrace{10}_{\text{right}} \} = 10$$

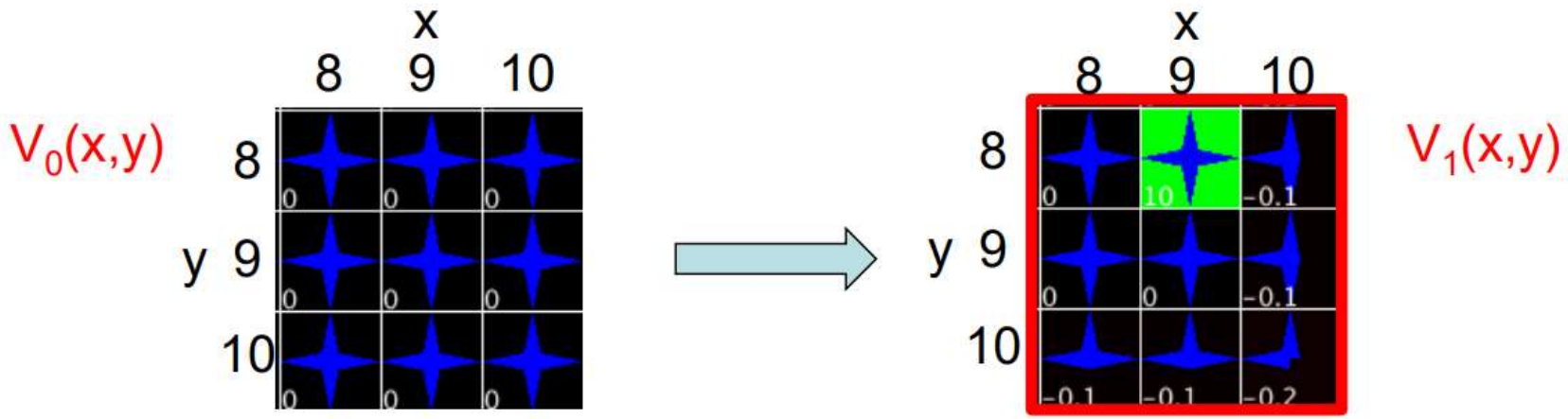


Value iteration example: Grid World

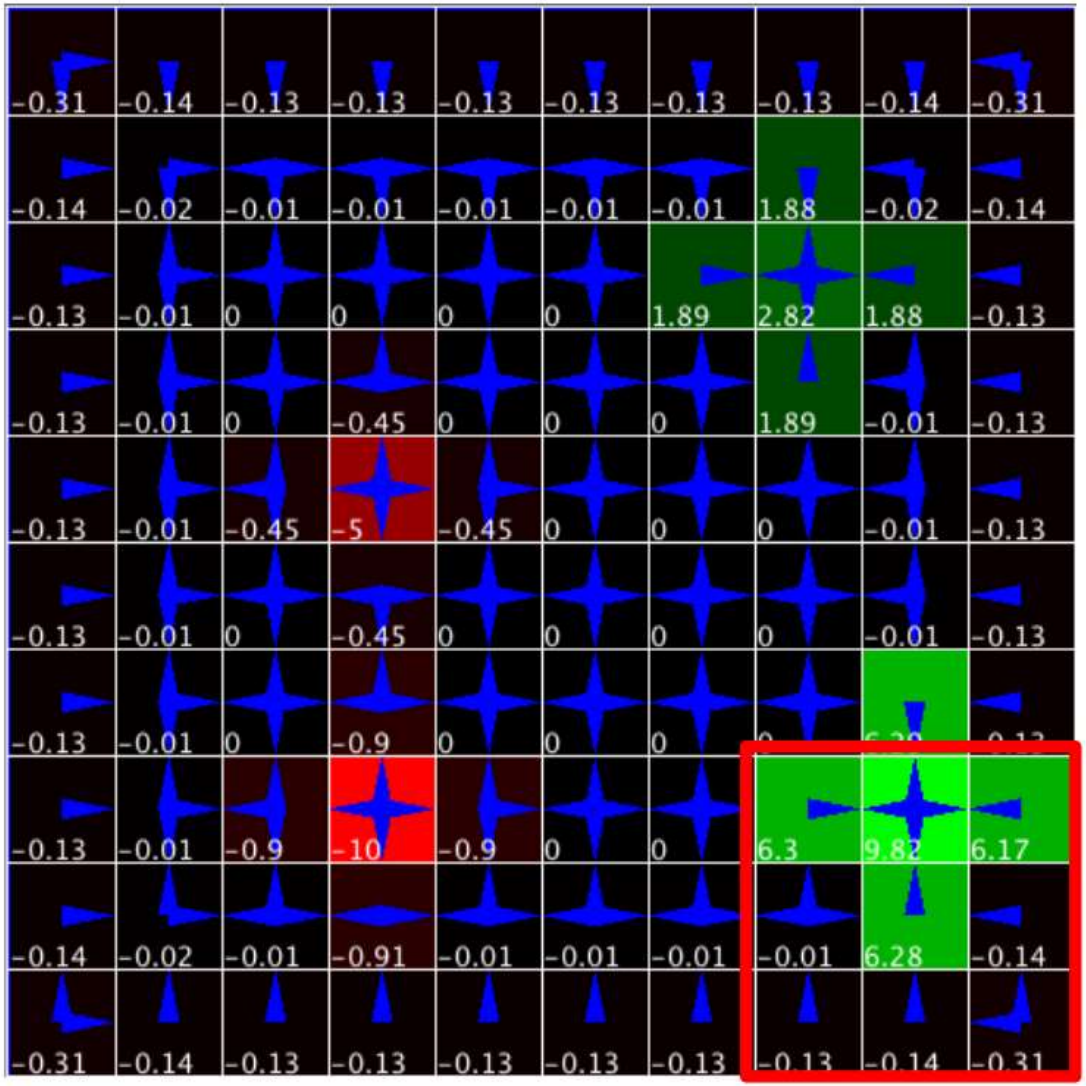
$$V_1^*(10,10) = \max_a \left[R(10,10,a) + \gamma \sum_{(x',y')} p(x',y' \mid 10,10,a) V_0^*(x',y') \right]$$

$$= \max_a R(10,10,a)$$

$$= \max_a \left\{ \underbrace{2 \cdot 0.1(-1)}_{\text{up}}, \underbrace{(0.7+0.1)(-1)}_{\text{down}}, \underbrace{2 \cdot 0.1(-1)}_{\text{left}}, \underbrace{(0.7+0.1)(-1)}_{\text{right}} \right\} = -0.2$$



Value iteration example: Grid World



$V_2(x,y)$ (Iteration 2)

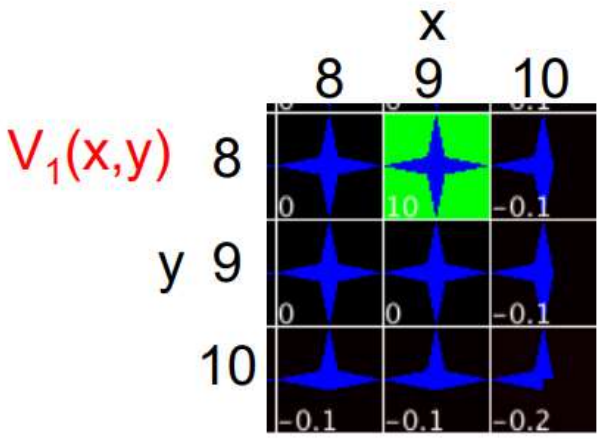
How are these calculated?

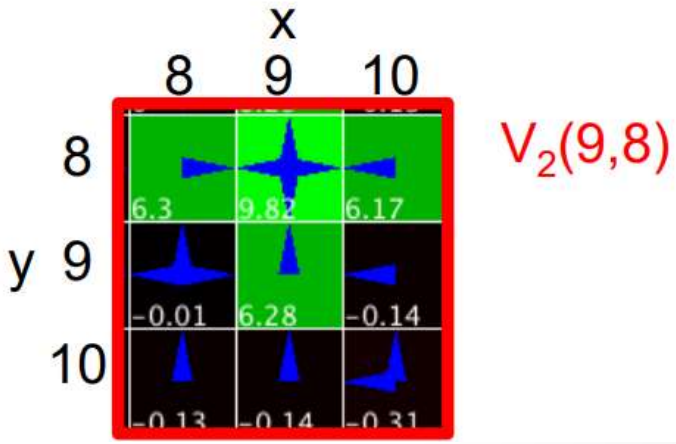
Value iteration example: Grid World

$$V_2^*(9,8) = \max_a \left[R(9,8,a) + \gamma \sum_{(x',y')} p(x',y' \mid 9,8,a) V_1^*(x',y') \right]$$

$$= 10 + 0.9 \times 0.25 \left[V(1,1) + V(1,10) + V(10,1) + V(10,10) \right]$$

$$= 10 + 0.9 \times -0.2 = 9.82$$



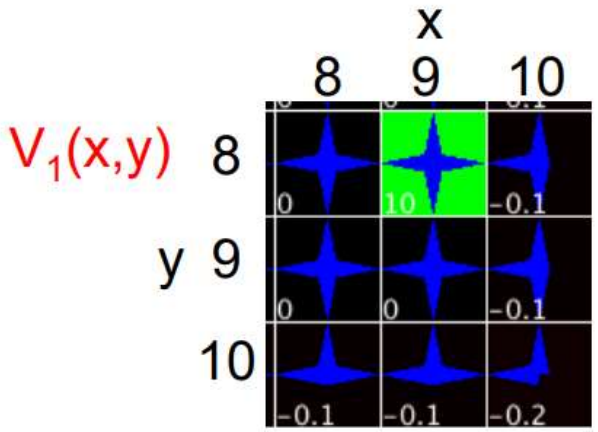


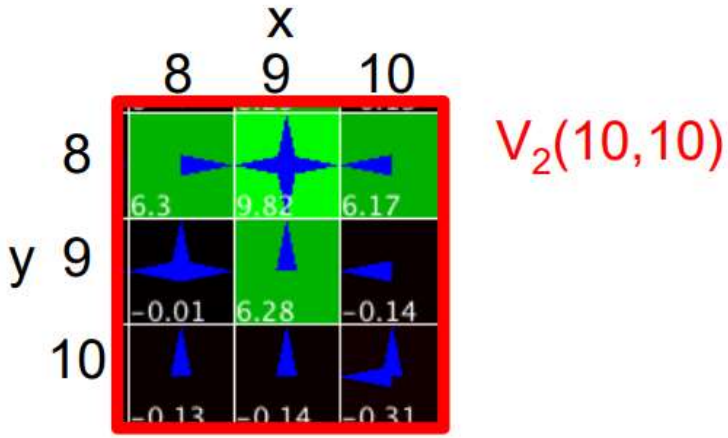
Value iteration example: Grid World

$$V_2^*(10,10) = \max_a \left[R(10,10,a) + \gamma \sum_{(x',y')} p(x',y' \mid 10,10,a) V_1^*(x',y') \right]$$

$$a = \text{up} \qquad -0.2 + 0.9\left[0.7(-0.1) + 0.1(-0.1) + 2{\cdot}0.1(-0.2)\right] = -0.31$$

$$a = \text{down} \qquad -0.8 + 0.9\left[(0.7 + 0.1)(-0.2) + 2{\cdot}0.1(-0.1)\right] = -0.96$$



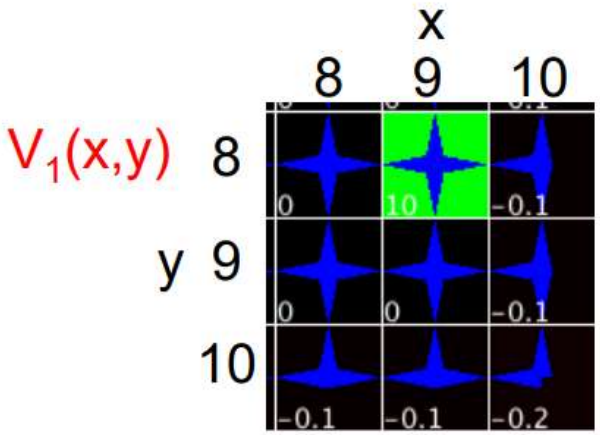


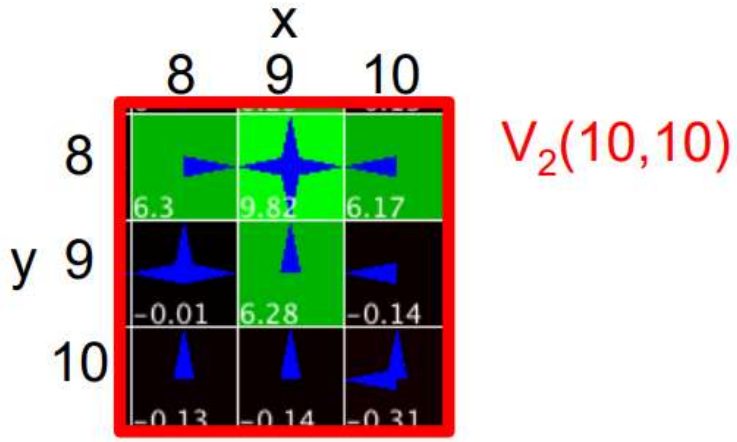
Value iteration example: Grid World

$$V_2^*(10,10) = \max_a \left[R(10,10,a) + \gamma \sum_{(x',y')} p(x',y' \mid 10,10,a) V_1^*(x',y') \right]$$

$$a = \text{left} \qquad -0.2 + 0.9\left[0.7(-0.1) + 0.1(-0.1) + 2{\cdot}0.1(-0.2)\right] = -0.31$$

$$a = \text{right} \qquad -0.8 + 0.9\left[(0.7 + 0.1)(-0.2) + 2{\cdot}0.1(-0.1)\right] = -0.96$$

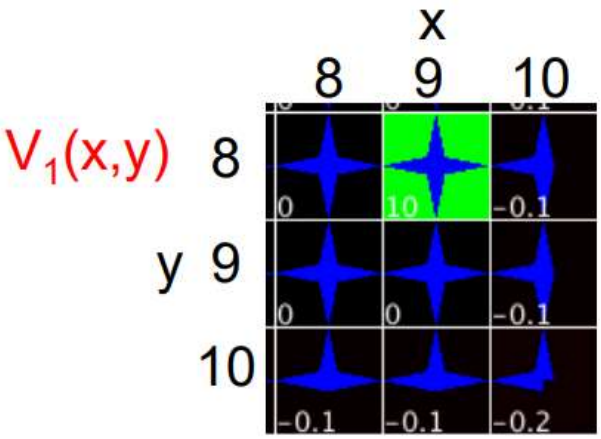


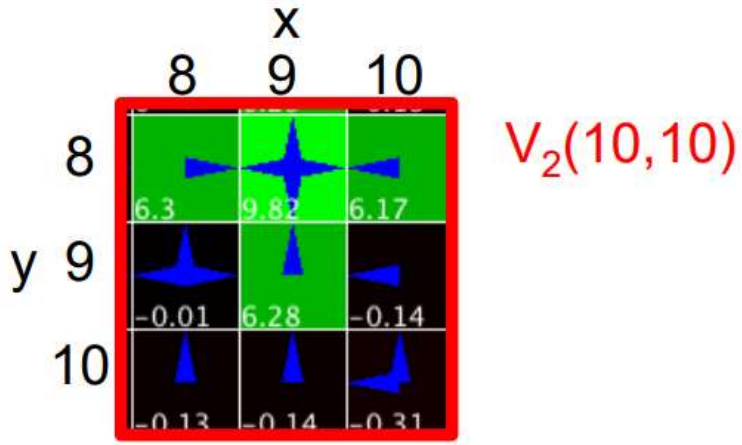


Value iteration example: Grid World

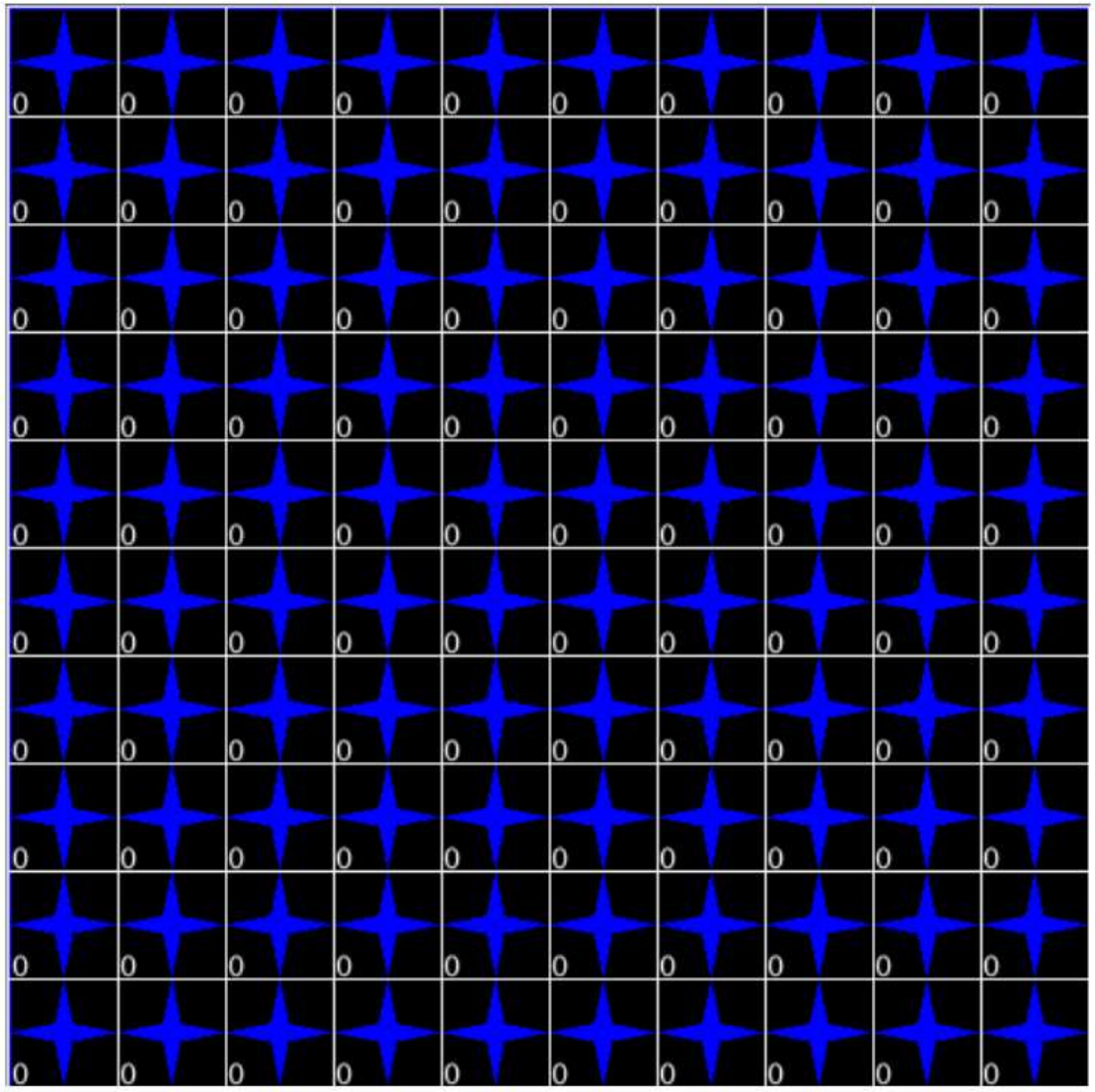
$$V_2^*(10,10) = \max_a \left[R(10,10,a) + \gamma \sum_{(x',y')} p(x',y' \mid 10,10,a) V_1^*(x',y') \right]$$

$$= -0.31$$



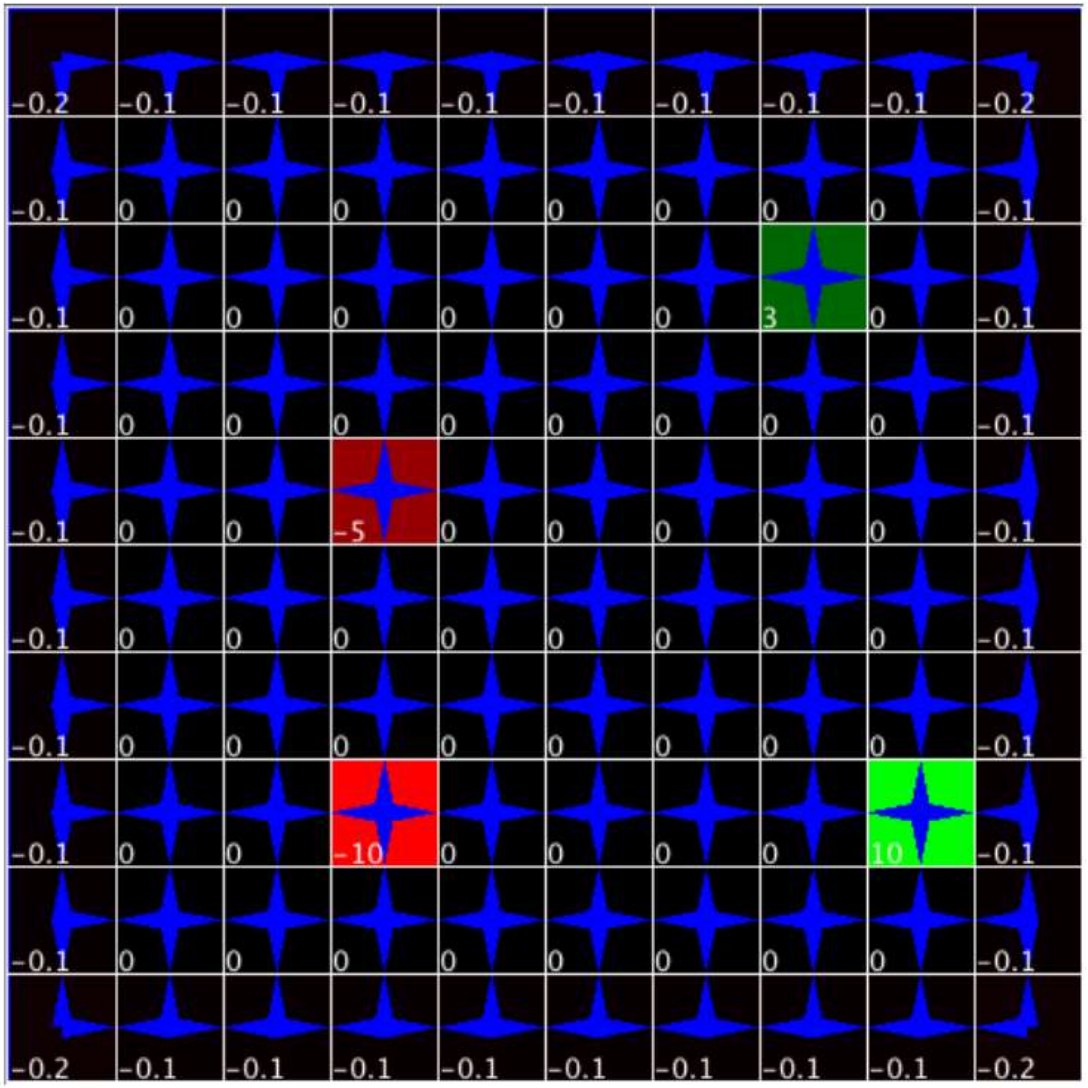


Value iteration example: Grid World



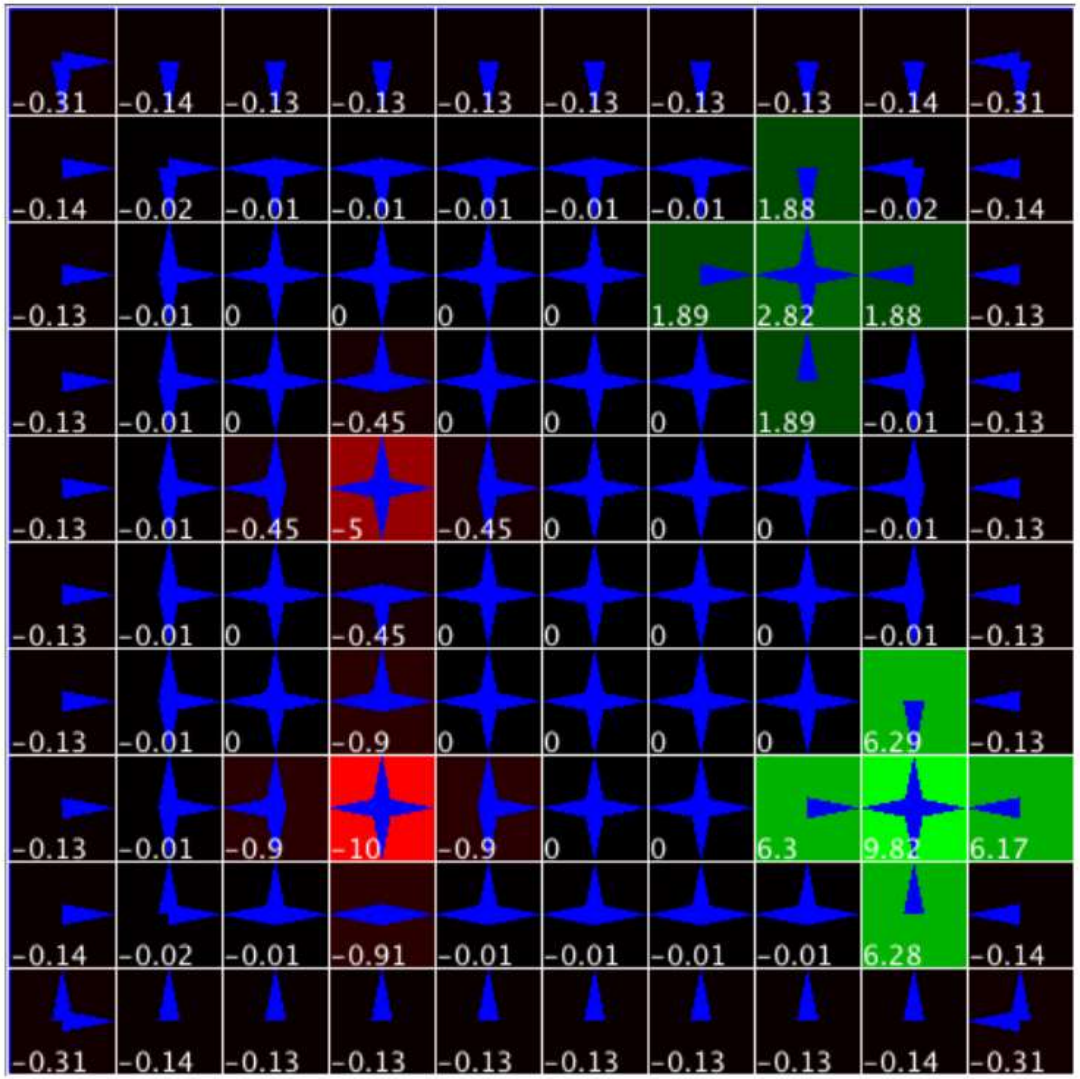
$V_0(x,y)$ (Iteration 0)

Value iteration example: Grid World



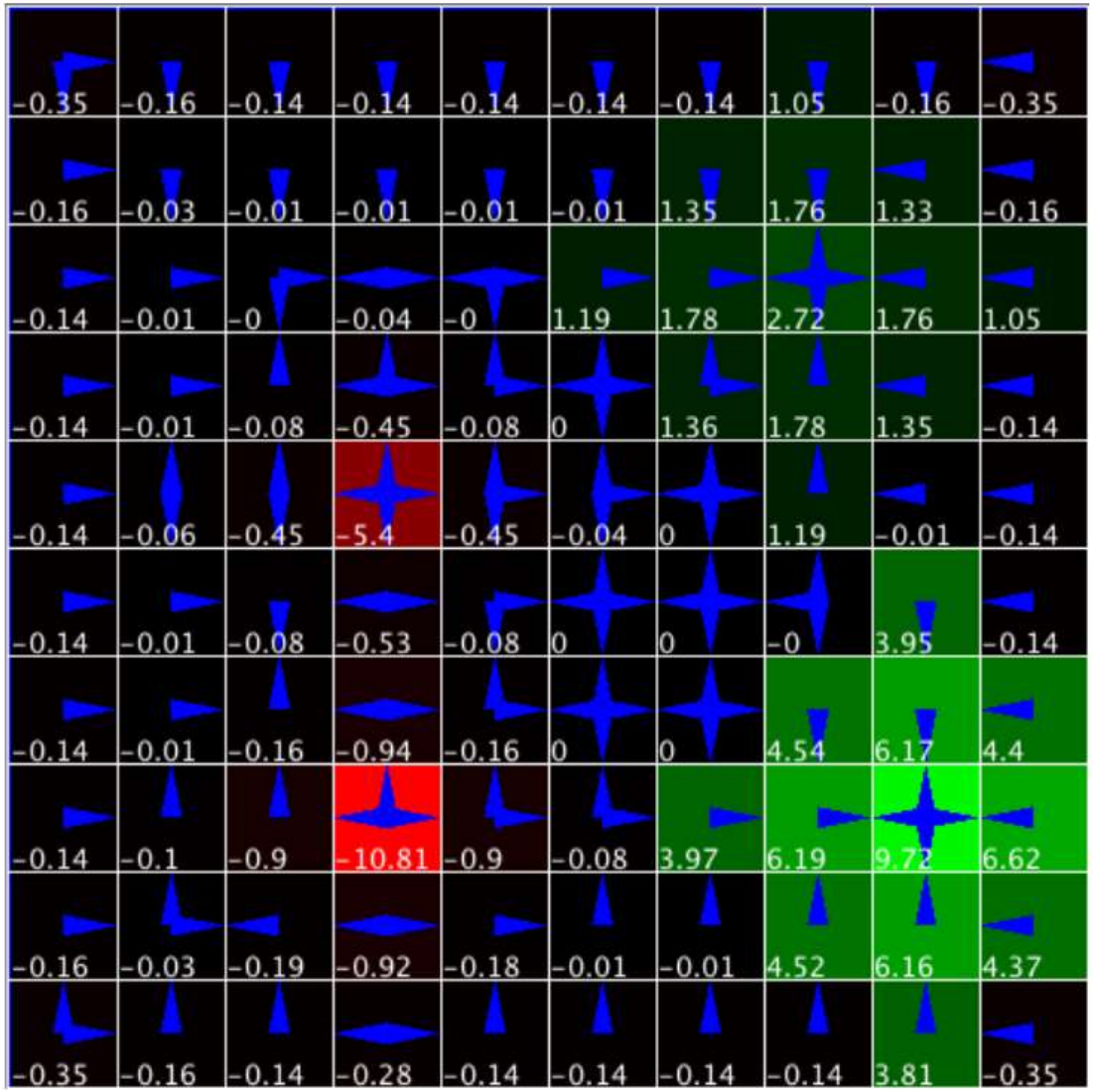
$V_1(x,y)$ (Iteration 1)

Value iteration example: Grid World



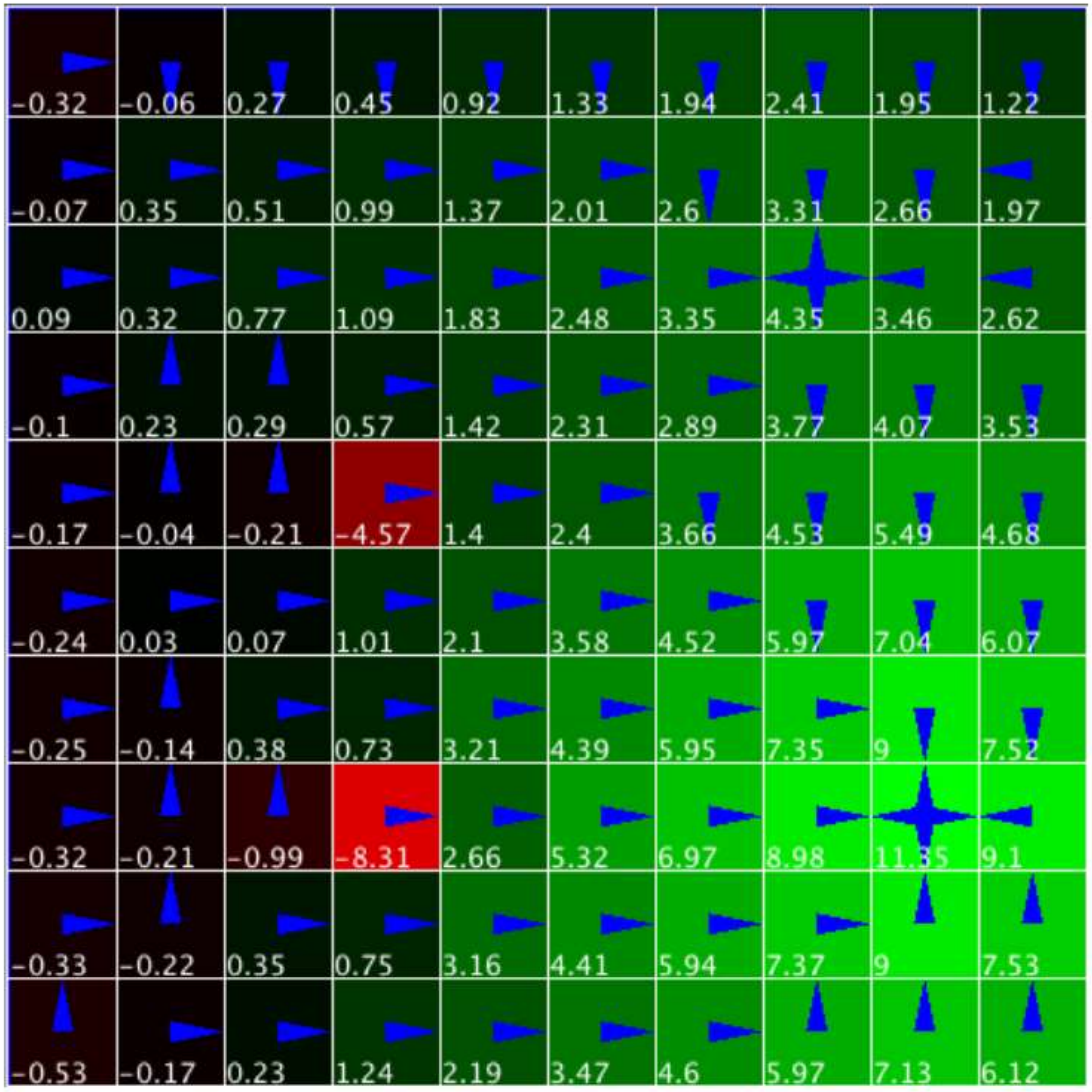
$V_2(x,y)$ (Iteration 2)

Value iteration example: Grid World



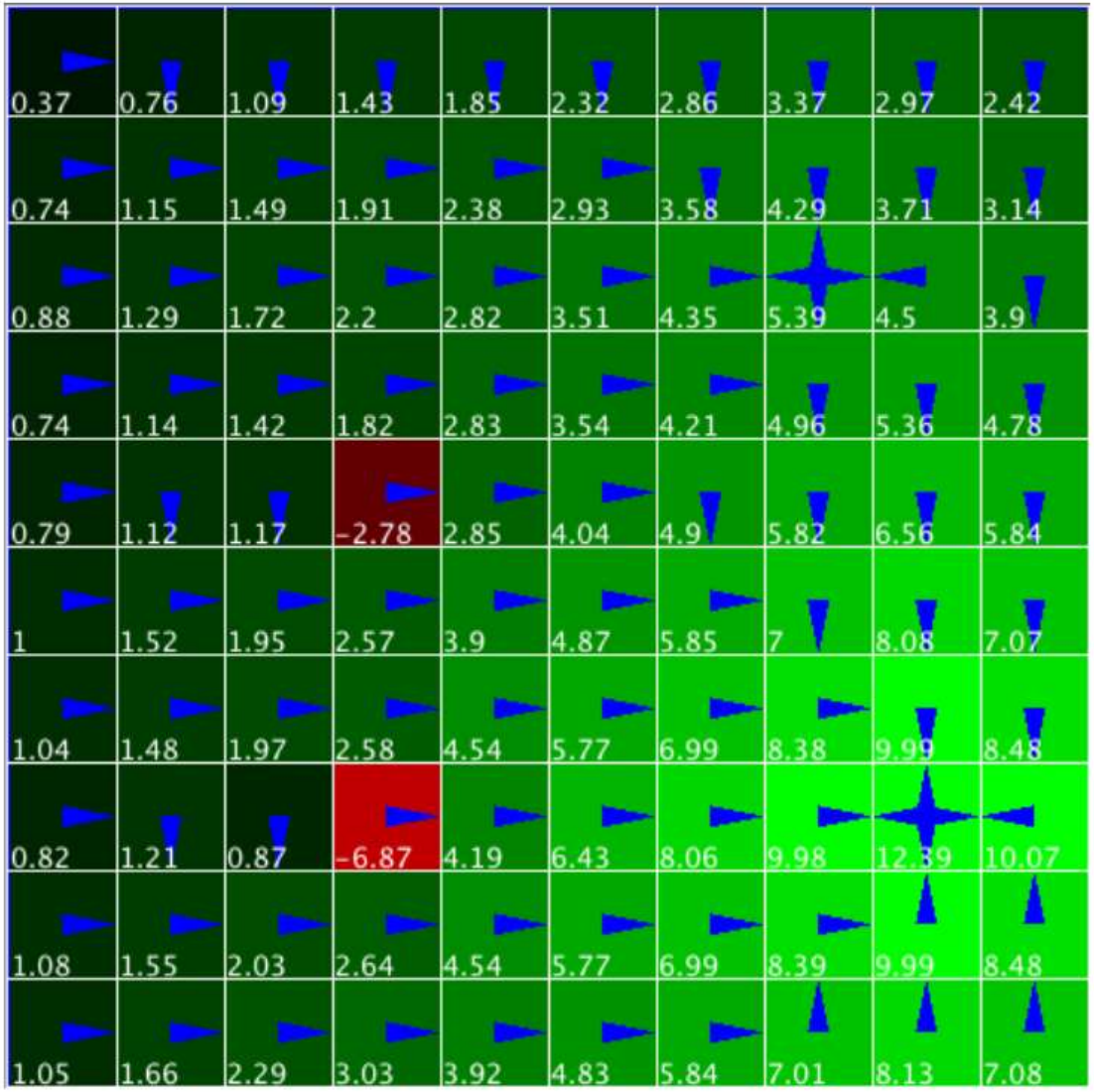
$V_2(x,y)$ (Iteration 3)

Value iteration example: Grid World



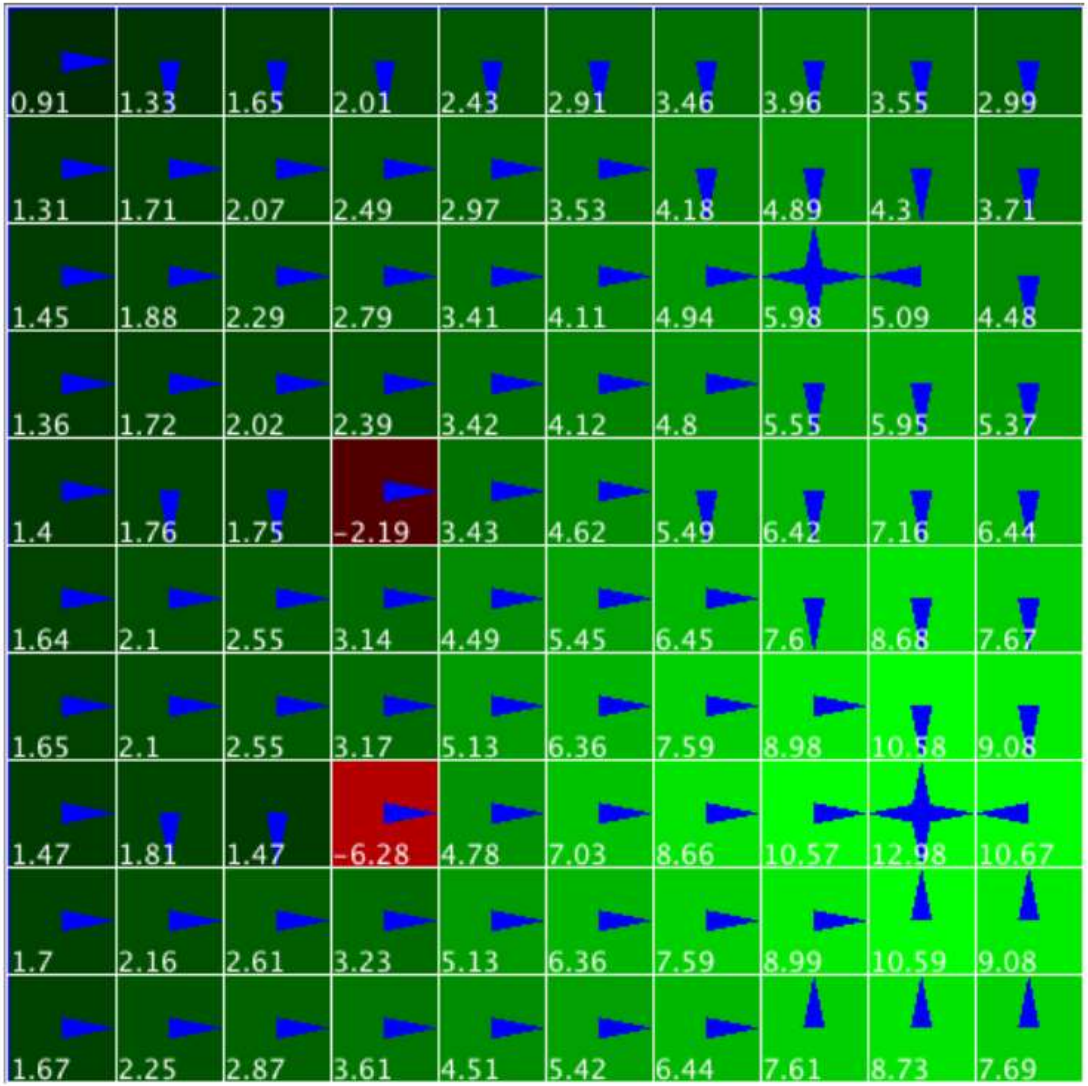
$V_{10}(x,y)$ (Iteration 10)

Value iteration example: Grid World



$V_{20}(x,y)$ (Iteration 20)

Value iteration example: Grid World



$V_{50}(x,y)$ (Iteration 50)

Difference between Policy Iteration and Value Iteration

1. Policy iteration includes: **policy evaluation** + **policy improvement**, and the two are repeated iteratively until policy converges.
2. Value iteration includes: **finding optimal value function** + **one policy extraction**. There is no repeat of the two because once the value function is optimal, then the policy out of it should also be optimal (i.e. converged).

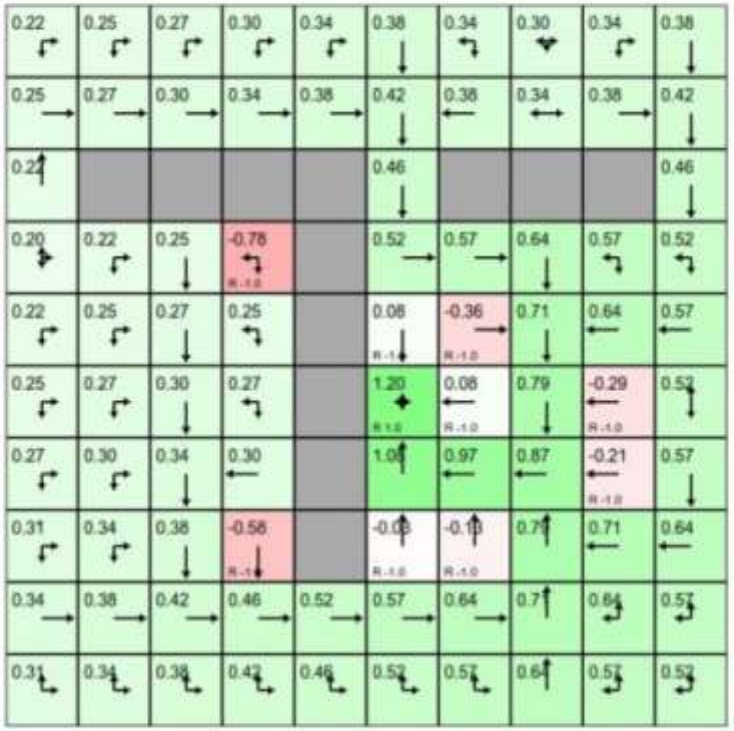
Decision Making in Markov Decision Process (MDP)

1. Prediction (evaluate a given policy):
   1. Input: MDP $< \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma >$ and policy π or MRP $< \mathcal{S}, \mathcal{P}^{\pi}, \mathcal{R}^{\pi}, \gamma >$
   2. Output: value function v^{π}
2. Control (search the optimal policy):
   1. Input: MDP $< \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma >$
   2. Output: optimal value function v^{*} and optimal policy π^{*}

Summary for Prediction and Control in MDP

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Demo of policy iteration and value iteration

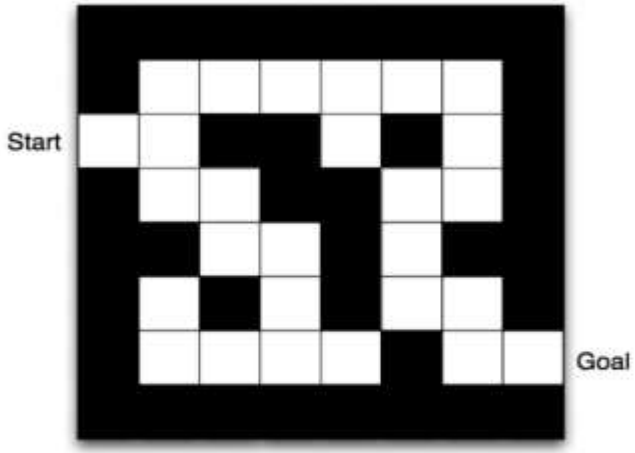


1. Policy iteration: Iteration of policy evaluation and policy improvement(update)
2. Value iteration

https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html

Assignments

- Solve the Maze Problem using Policy Iteration or Value Iteration (实验报告3页以上：MDP建模、算法说明、实验结果等)



- Rewards: −1 per time-step
- Actions: N, E, S, W
- States: Agent's location

- *提交邮箱：rlhomework2022@163.com*
- *邮件+文件命名：学号+姓名*
- *提交期限：11月5号23:59*